IERG4320/IEMS5726: Data Science in Practice

Lecture 12: Case Studies (IV) Game Problems

Prof. YAN Wenjing, Dr. Danny Yip
Department of Information Engineering

香港中文大學
The Chinese University of Hong Kong

Outline

1. Reinforcement Learning in card game or board game
   - D3QN, A3C and AlphaZero → AlphaBlackjack
2. Single agent for multiple games
   - Elastic Weight Consolidation
3. 3D Model Generation
   - 2D-to-3D: TripoSR
   - Text-to-3D: Shap-E
4. Robot Learning

All rights are reserved to Prof. YAN Wenjing, Dr. Danny YIP,
The Department of Information Engineering, CUHK

Types of Game Genres

1. Party Games
   - Pictionary
2. Tabletop Games
   - Card games
   - Board games
3. Video (or console) Games
   - Role-Play Games (RPG)
   - Soul games
4. Casual and Mini Games
   - Agar.io
   - Watermelon game

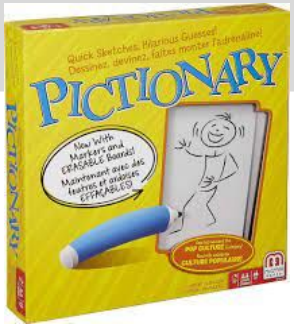


Image Credit: Google Image

Games and AI / ML

- Observations from games:

1. In general, not much game data to learn and process
2. Well-defined game settings and game rules (**environment**)
3. Well-defined gameplay (**actions**)
4. Well-defined goals (**rewards**)
5. Different ways of matching (**agents**):
    - Single-player game, multiplayer game, … etc.

- Insight: We perform searching based on the game rules, and identify the best strategy to win the game.
- *Disclaimer: There are always exceptions (e.g. survival game)*

Roles of AI in Games

1. Opponents
2. Teammates
3. Strategic Opponents
4. Support Characters
5. Autonomous Characters
6. Commentators
7. Camera Control
8. Plot and Story Guides/Directors

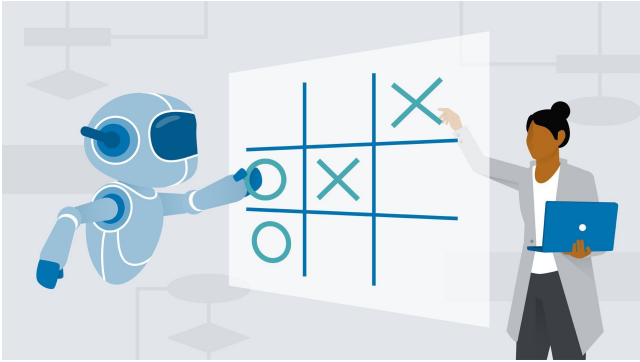


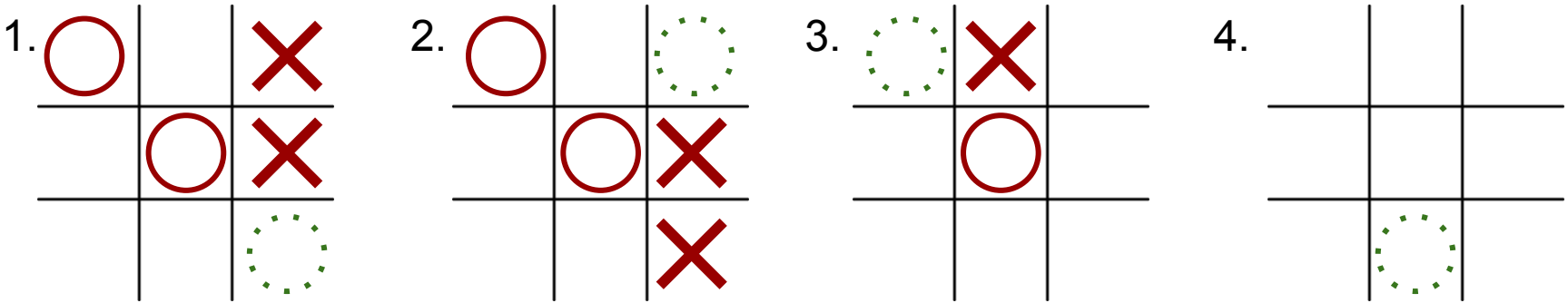
Image credit: Google Image

Rule-Based System

- Rule-based systems are a type of AI that relies on a set of rules to make decisions.
- There is a clear set of rules to follow, such as in a game of chess.
- Rule-based systems can be limited in their ability to deal with more complex situations.

Example: A Simple Rule-Based System for Tic-Tac-Toe

1. If the agent has a winning move, then win the game.
2. If the opponent has a winning move, then block the opponent.
3. If the agent can promote a winning move in the next step, then place that check.
4. Pick a random check to place.



Other Common Searching Techniques in AI

1. **A* Search**: A graph traversal and path search algorithm
2. **Alpha-beta Pruning**: A search algorithm that seeks to decrease the number of nodes that are evaluated by the minimax algorithm in its search tree.
3. **Fuzzy logic expert system**: A fuzzy expert system is a collection of fuzzy rules and membership functions that are used to reason data.
   - Example of fuzzy logic: Degree of membership within the set of old people is 0.90 → There is a 90% chance that a member is old
4. **Genetic algorithm**: A method for solving both constrained and unconstrained optimization problems that is based on natural selection, the process that drives biological evolution.
5. … etc

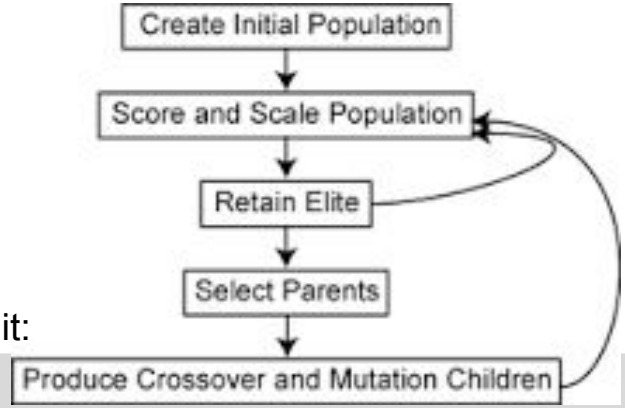
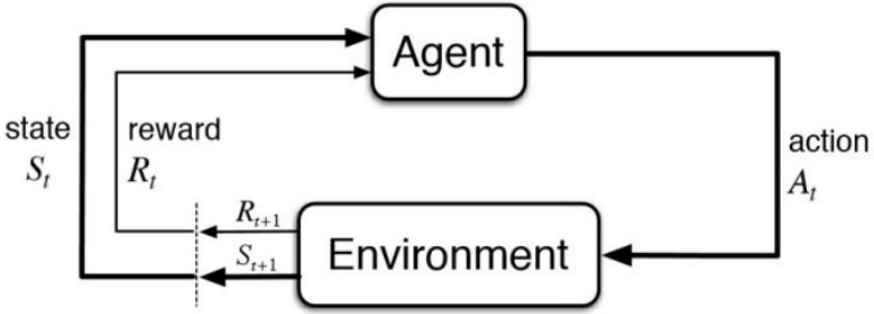


Image credit: Mathworks

Introduction to Reinforcement Learning (Revisit)

- Machine learning through interactions in simulation

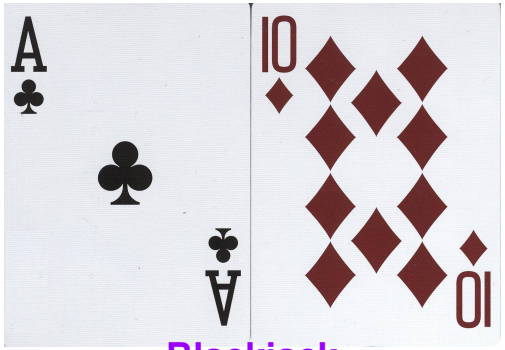


- A computational approach to learning whereby **an agent** tries to **maximize** the total amount of **reward** it receives while interacting with a complex and uncertain **environment**.
  - Reference: Sutton and Barto Book

Brief Introduction to RL: https://www.youtube.com/watch?v=JgvyzIkgxF0

Blackjack

- Objective: Beat the dealer's hand without going over 21
  In other words, get a hand that's higher than the dealer's
  - **2 through 10**: The number listed on the card (e.g., 2 is worth 2 points)
  - **Jack, Queen, King**: 10 points
  - **Ace**: 1 or 11 points (the player gets to choose)



Blackjack

- Special rule: An Ace and a 10 / Jack / Queen / King is called as a **Blackjack**.
  Blackjack is the strongest hand, higher than 3 or more cards with sum = 21.

Blackjack Game Flow

1. All the players make a bet with their chips.
2. The dealer gives a card to each player as well as themselves.
3. The dealer hands out a second card to every player.
4. The player to the left of the dealer starts gameplay.
5. Player decisions.
    - Hit: Take another card
    - Stand: Take no more cards
    - Double down: Increase the initial bet by 100% and take exactly one more card
6. Cycle through each player until they've each finished their turn.
7. The dealer reveals their second card or more and winners are determined.
    - If the total is 16 or lower, they're required to hit and take another card.
    - If the card is 17 or higher, the dealer is required to stay.

Blackjack Environment - State

- The state is the information available to the agent to make a decision. The state is a **tuple** of three elements.
  - The **player's** current **sum** (e.g., 12, 19, 21).
  - The **dealer's** showing **card** (e.g., 2 through Ace, where Ace is 11).
  - Whether the player has a **usable Ace**. A "usable" Ace is an Ace that can be counted as 11 without the player busting (going over 21). This is a boolean value (`True`/`False`).
- For example:
  - `(15, 10, False)`: Player has 15, dealer shows a 10, no usable Ace.
  - `(13, 5, True)`: Player has 13, dealer shows a 5, has a usable Ace (meaning the hand could also be counted as a soft 3).
  - `(21, 7, False)`: A natural blackjack or a hard 21.

Blackjack Environment - Action

1. **Hit**: Take another card
2. **Stand**: Take no more cards

- We do not include **double down** and **split** in this example.

Blackjack Environment - Reward

- The reward is received at the end of the episode (hand).

1. `+1`: If the player wins.
2. `-1`: If the player loses.
3. `0`: If the game is a draw (push).

- As Blackjack is a classic and standard example, there is no intermediate reward.
- The agent only knows if it won or lost after the hand is complete.
- This sparse reward signal is a key characteristic of the Blackject problem.

Blackjack Environment - State Transition

- This defines how the environment moves from one state to the next given an action.

1. If action is **HIT**: The environment draws a card uniformly at random from the deck (with replacement in the infinite deck assumption). The player's sum is updated. If the new sum exceeds 21, the player busts and the episode terminates with a reward of -1. If the player has a usable Ace and busts, the Ace converts from 11 to 1, and the game continues.
2. If action is **STAND**: The player's turn ends. The dealer then reveals their hidden card and follows a fixed policy (the "dealer's policy").

Blackjack Environment - Fixed Dealer’s Policy

- The dealer must HIT until their sum is 17 or greater.
- In most implementations, if the dealer has a usable Ace (a “soft 17”), they must also HIT.

Example: Episode

1. Initial State: Player is dealt (8, 10). Dealer shows a 9.
   - State: `(8, 9, False)`

2. Agent's Turn:
   - Action: **HIT** (the agent decides it needs a higher sum).
   - New Card: 7
   - New State: `(15, 9, False)`
   - Action: **HIT**
   - New Card: 3
   - New State: `(18, 9, False)`
   - Action: **STAND** (the agent is satisfied with 18).

Example: Episode

3. Dealer’s Turn:
   - Dealer reveals hidden card: 6. Dealer's sum is 15 (9 + 6).
   - Dealer must **HIT** (sum < 17).
   - Dealer draws: 10.
   - Dealer's new sum is 25 → BUST.

4. Termination & Reward:
   - The player wins. The agent receives a reward of **+1** for the entire episode.

Some Probabilities about Blackjack

- If we use a basic strategy to play Blackjack
  - Win rate: 42.22%
  - Loss rate: 49.10%
  - Tie: 8.48%

- If we skip ties
  - Win rate: 46.36%
  - Loss rate: 53.64%

- Reference: https://wizardofodds.com/ask-the-wizard/blackjack/probability/

Dueling Double Q Network (D3QN)

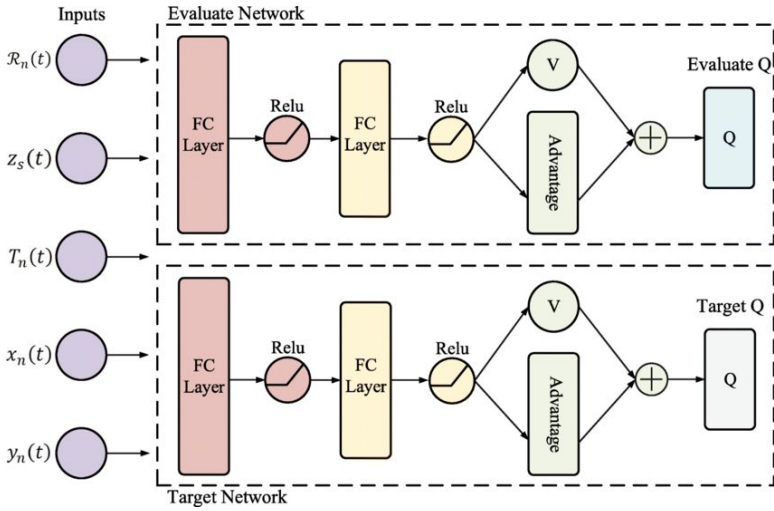
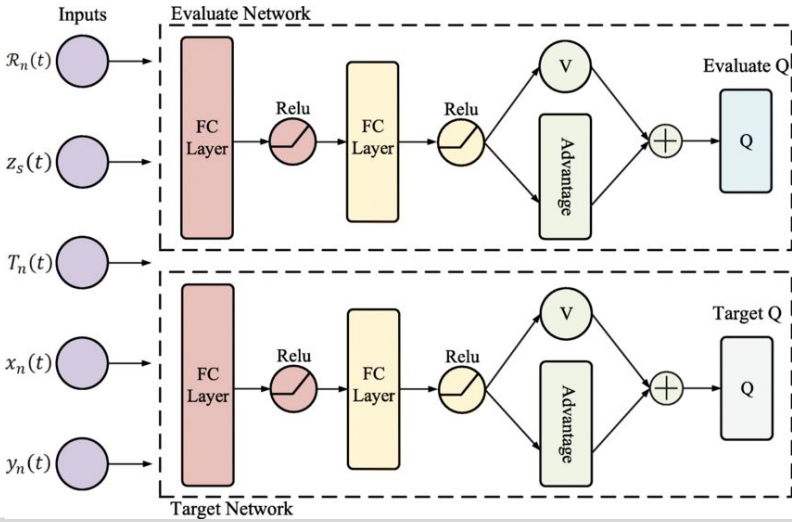


Image credit: ResearchGate
Code:
https://colab.research.google.com/drive/1XUTXZxB4hvhqG5ynKGMOvloIkSqppFgw#scrollTo=xBJMZ3NWXtpf

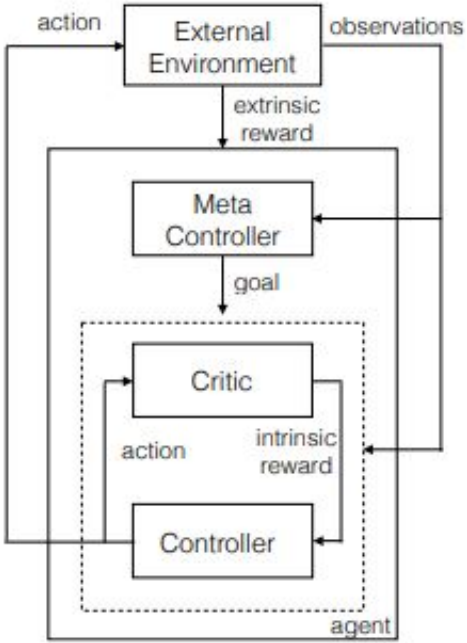
Introduction to Dueling Double Q Network (D3QN)

1. **Double DQN**: Addresses the overestimation bias (max operator) in Q-values by using separate networks for action selection and evaluation.
   - Standard DQN: $y = r + \gamma * \max_{a'} Q\square_{ar}(s',a'; \theta^-)$
   - Double DQN: $y = r + \gamma * Q\square_{ar}(s', \text{argmax}_{a'} Q(s',a'; \theta); \theta^-)$
   - Example: States like `(16, 10, False)` have very similar Q-values for both actions
   - Double DQN provides more realistic value estimates



Deep Hierarchical Reinforcement Learning

- The agent produces actions and receives sensory observations.
- Separate deep-Q networks are used inside the **meta-controller** and **controller**.
- The meta-controller that looks at the raw states and produces a policy over goals by estimating the value function by maximizing expected future extrinsic reward.
- The controller takes in states and the current goal, and produces a policy over actions by estimating the value function to solve the predicted goal by maximizing expected future intrinsic reward.
- The internal critic checks if the goal is reached and provides an appropriate intrinsic reward to the controller.
- The controller terminates either when the episode ends or when the goal is accomplished.
- The meta-controller chooses a new goal and the process repeats.



Introduction to Dueling Double Q Network (D3QN)

2. **Dueling Architecture**: Splits the Q-network into two streams:
   - **Value stream**: Estimates how good the state is (V(s))
   - **Advantage stream**: Estimates how much better each action is compared to others (A(s,a))
   - Combined as:
     i. **Mean Advantage**: $Q(s,a) = V(s) + (A(s,a) - \text{mean}_a(A(s,a)))$
     ii. **Max Advantage**: $Q(s,a) = V(s) + (A(s,a) - \max_a(A(s,a)))$
   - Example: state = (20, 2, False)
     i. Value V(s): Very high - we're in a great position regardless of action
     ii. Advantage A(s,STAND): Slightly positive - standing is safe
     iii. Advantage A(s,HIT): Very negative - hitting risks busting

Introduction to Dueling Double Q Network (D3QN)

3. **Deep Q-Network**: Uses neural networks to approximate Q-values for high-dimensional state spaces.
   - $\text{Loss} = \mathbb{E}[(r + \gamma * Q\square_{ar}(s', \text{argmax}_{a'} Q(s',a'; \theta); \theta^{-}) - Q(s,a; \theta))^2]$
   - θ = parameters of policy network (online network)
   - θ^{-} = parameters of target network (updated periodically)
   - Q-value: $Q(s,a; \theta) = V(s; \theta) + A(s,a; \theta)$

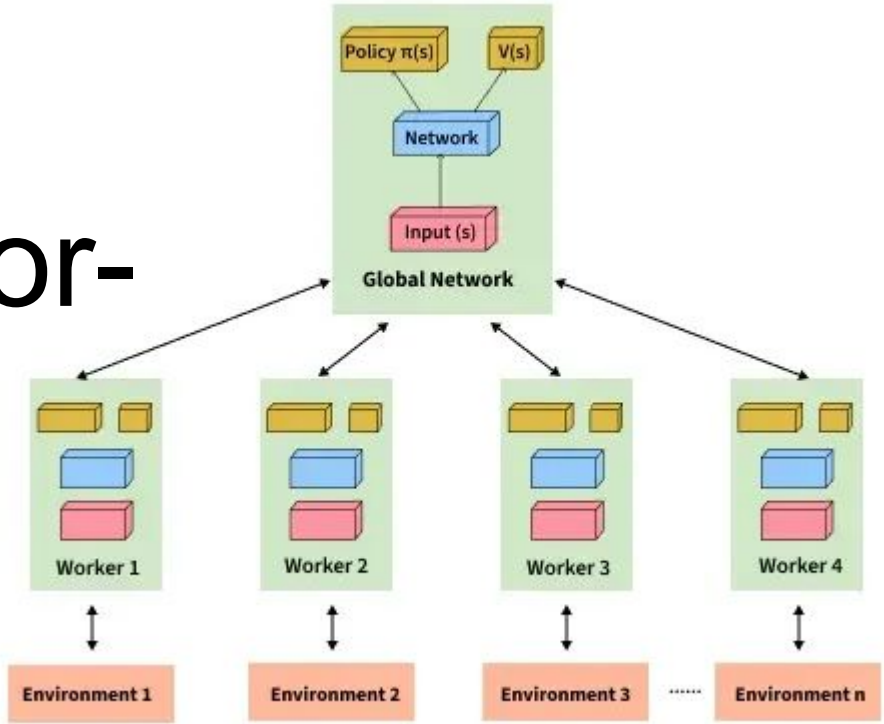
After playing 10000 games, the win rate stabilizes around 42% - 44%

```
Starting D3QN training for Blackjack...
Episode 0, Epsilon: 1.000, Recent Win Rate: 0.000
Episode 1000, Epsilon: 0.010, Recent Win Rate: 0.370
Episode 2000, Epsilon: 0.010, Recent Win Rate: 0.390
Episode 3000, Epsilon: 0.010, Recent Win Rate: 0.440
Episode 4000, Epsilon: 0.010, Recent Win Rate: 0.390
Episode 5000, Epsilon: 0.010, Recent Win Rate: 0.380
Episode 6000, Epsilon: 0.010, Recent Win Rate: 0.460
Episode 7000, Epsilon: 0.010, Recent Win Rate: 0.480
Episode 8000, Epsilon: 0.010, Recent Win Rate: 0.390
Episode 9000, Epsilon: 0.010, Recent Win Rate: 0.430

Evaluating trained agent...
Evaluation Results (10000 episodes):
Win Rate: 0.434
Loss Rate: 0.478
Draw Rate: 0.088
Average Reward: -0.044
```

Asynchronous Advantage Actor-Critic (A3C)

Image credit: GeekForGeek
Code:
https://colab.research.google.com/drive/1XUTXZxB4hvhqG5ynKGMOvloIkSqppFgw#scrollTo=OH2pEbKMsqqp



Limitations for DQN

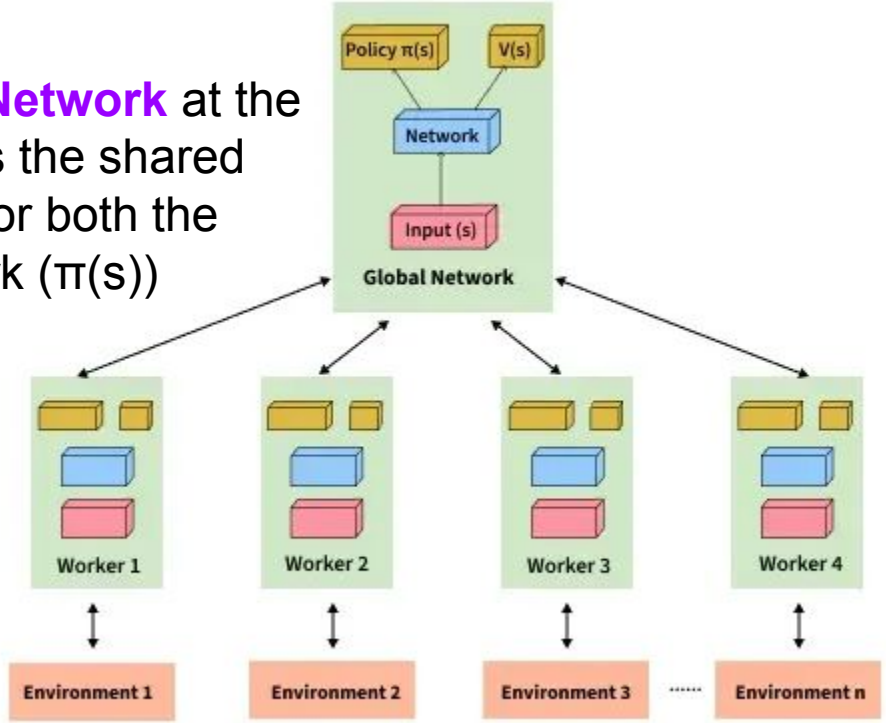
1. **High Correlation**: Experience replay uses a correlated sequence of experiences, which is inefficient for neural networks.
2. **High Resource Consumption**: DQN and other algorithms required powerful GPUs and a lot of memory for the experience replay buffer.
3. **Lack of Generalization**: They were often specialized for a single task.

- Key differences between DQN and A3C
- DQN: Uses discrete action spaces and suffers from maximization bias
- A3C: Supports continuous actions, trains faster and avoids Q-learning's bias

Asynchronous in A3C

- Instead of one agent with one replay buffer, use multiple agent workers in parallel.

The **Global Network** at the top maintains the shared parameters for both the policy network (π(s)) and value network (V(s))



*Each **worker** explores different parts of the state space, decorrelating the data naturally.*

Image credit: GeekForGeek

Asynchronous Training

A3C runs several agents in parallel, each interacting independently with a separate copy of the environment. These workers collect experience at different rates and send updates simultaneously to a central global network. This parallelism helps:

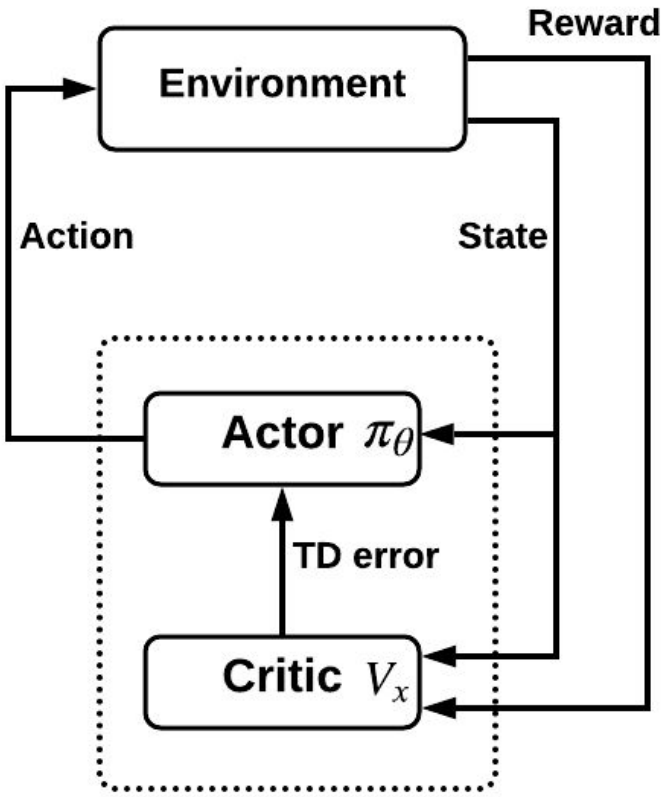
1. Speed up training
2. Provide diverse experience to avoid overfitting
3. Reduce sample correlation (a common issue in reinforcement learning)

Reference:
https://www.geeksforgeeks.org/machine-learning/asynchronous-advantage-actor-critic-a3c-algorithm/

Actor-Critic Framework

- The actor is responsible for action selection, while the critic evaluates those actions to help improve the policy.

1. **Actor**: Learns the policy π(a|s) which defines the probability of taking action a in state s.
2. **Critic**: Learns the value function V(s) which estimates how good a given state is.



Advantage Function

- A3C incorporates the advantage function: $A(s,a)=Q(s,a)-V(s)$

1. Provide clearer learning signals
2. Reduce the variance in policy gradient updates.

Mathematical Intuition for A3C

- The advantage function plays an important role in A3C. When an agent takes an action a in state s, the advantage function tells us whether the reward is better than expected:
  - Positive advantage → reinforce this action
  - Negative advantage → discourage this action.
- A3C uses n-step returns to strike a balance between bias and variance:
  - Shorter n → more bias, less variance (quicker updates),
  - Longer n → less bias, more variance (smoother updates).
- The learning objectives are:
  - Actor: Increase the probability of actions with higher advantage,
  - Critic: Reduce error in value prediction for better advantage estimation.

How A3C Works: Training Pipeline

1. **Experience Collection**: Each worker interacts with its environment and collects sequences of states, actions and rewards.
2. **Advantage Estimation**: Workers compute n-step returns and use them to estimate the advantage of each action taken.
3. **Gradient Calculation**: Using the computed advantages, each worker calculates gradients for both the actor and critic networks.
4. **Asynchronous Update**: Workers update the shared global network independently, without waiting for other workers to complete their updates.

After playing 10000 games, the win rate stabilizes around 38% - 40%

```
=== A3C Blackjack Training ===
Worker 0 started
Worker 1 started
Worker 2 started
Worker 3 started
Worker 1, Episode 0: Reward=-1.0, Avg=-1.000, WinRate=0.000
Worker 2, Episode 0: Reward=0.0, Avg=0.000, WinRate=0.000
Worker 0, Episode 0: Reward=-1.0, Avg=-1.000, WinRate=0.000
Worker 3, Episode 0: Reward=-1.0, Avg=-1.000, WinRate=0.000
Worker 2, Episode 500: Reward=1.0, Avg=-0.250, WinRate=0.340
Worker 1, Episode 500: Reward=-1.0, Avg=-0.200, WinRate=0.360
Worker 3, Episode 500: Reward=1.0, Avg=-0.280, WinRate=0.350
Worker 0, Episode 500: Reward=-1.0, Avg=-0.170, WinRate=0.390
Worker 1, Episode 1000: Reward=1.0, Avg=-0.040, WinRate=0.450
Worker 3, Episode 1000: Reward=1.0, Avg=-0.280, WinRate=0.340
Worker 2, Episode 1000: Reward=-1.0, Avg=-0.250, WinRate=0.350
Worker 0, Episode 1000: Reward=1.0, Avg=-0.170, WinRate=0.380
Worker 3, Episode 1500: Reward=-1.0, Avg=-0.350, WinRate=0.300
Worker 1, Episode 1500: Reward=-1.0, Avg=-0.170, WinRate=0.410
Worker 2, Episode 1500: Reward=1.0, Avg=-0.260, WinRate=0.330
Worker 0, Episode 1500: Reward=1.0, Avg=-0.260, WinRate=0.350
```

```
Worker 3, Episode 2400: Reward=-1.0, Avg=-0.300, WinRate=0.340
Worker 1, Episode 2400: Reward=-1.0, Avg=-0.050, WinRate=0.460
Worker 0, Episode 2400: Reward=-1.0, Avg=-0.180, WinRate=0.400
Worker 2, Episode 2400: Reward=-1.0, Avg=-0.110, WinRate=0.420

Training completed in 51.03 seconds
Final 100 episodes - Avg reward: -0.100, Win rate: 0.410

Evaluating over 2000 episodes...
Results: Wins=771, Losses=1140, Draws=89
Win rate: 0.386
Stand rate: 1.000
Action distribution: Stand=2000, Hit=0

Analyzing learned policy:
State (12, 2, False): STAND (stand=0.999, hit=0.001, value=-0.250)
State (12, 9, False): STAND (stand=1.000, hit=0.000, value=-0.337)
State (16, 7, False): STAND (stand=1.000, hit=0.000, value=-0.124)
State (18, 9, False): STAND (stand=1.000, hit=0.000, value=0.002)
State (20, 5, False): STAND (stand=1.000, hit=0.000, value=0.500)
State (11, 2, False): STAND (stand=0.999, hit=0.001, value=-0.257)
State (8, 6, False): STAND (stand=0.999, hit=0.001, value=-0.301)

Model saved successfully!
```

AlphaZero
→ AlphaBlackjack

Image credit: GeekForGeek
Code:
https://colab.research.google.com/drive/1XUTXZxB4hvhqG5ynKGMOvloIkSqppFgw#scrollTo=6Yy5LSqQ0m1h

Introduction to AlphaZero

- AlphaZero was developed by the artificial intelligence and research company DeepMind, which was acquired by Google.
- AlphaZero is a computer program that reached a virtually unthinkable level of play using only reinforcement learning and self-play in order to train its neural networks.
- AlphaZero was only given the rules of the game and then played against itself many millions of times (44 million games in the first nine hours, according to DeepMind).

Monte Carlo Tree Search (MCTS)

- MCTS allows AlphaZero to efficiently process and anticipate game developments, combining depth of analysis with broad strategic exploration.

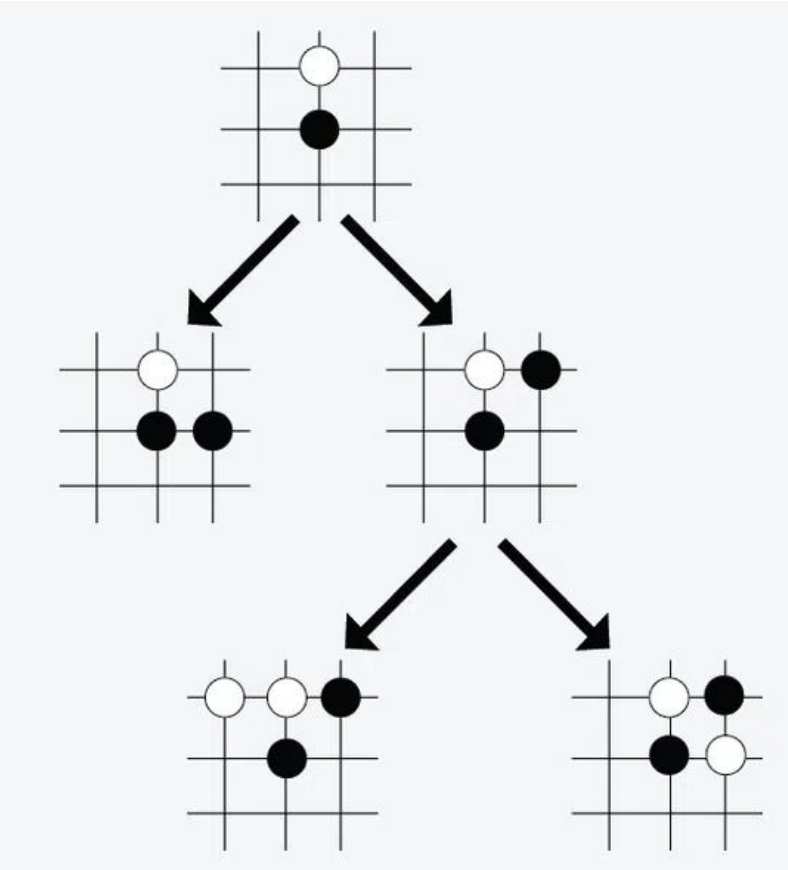


Image credit: https://www.geeksforgeeks.org/artificial-intelligence/alphago-algorithm-in-artificial-intelligence/

Monte Carlo Tree Search (MCTS)

1. **Selection**: MCTS uses the formula Q+U to select the most promising move path in the tree, considering both the quality of moves (Q) and the importance of exploring new moves (U).
   - Exploitation: Nodes that have historically led to wins
   - Exploration: Nodes that haven't been visited much
2. **Expansion**: New moves are explored by adding them to the tree, allowing the algorithm to evaluate a broader range of possibilities.
   - From a partially expanded node, we try a new action `(HIT or STAND)`
   - Create a new node representing the resulting game state
   - E.g., if current state is `(16, 7, False)`, we might expand by hitting to reach `(19, 7, False)`

Monte Carlo Tree Search (MCTS)

3. **Simulation**: The outcomes of possible moves are simulated using a simpler policy, typically the rollout policy, which quickly predicts move outcomes to expand the tree further.
   - From the expanded node, we play randomly until the game ends
   - Use a simple policy like “hit until 17, then stand”
   - Return the final game outcome (+1 for win, -1 for loss, 0 for draw)
4. **Backpropagation**: Results from the simulations are used to update the statistics of the moves in the tree, refining the strategy based on the outcomes of these simulations.
   - Propagate the simulation result back up the tree
   - Update visit counts and win statistics for all nodes along the path
   - This reinforces good sequences of moves

Dual-Headed Network Design

1. **Policy** Head - The "**Actor**"
   - **Purpose**: Learn which actions are best in each state
   - **Output**: Probability distribution over actions `[P(STAND), P(HIT)]`
   - **Training**: Improved using policy gradient with advantages
2. **Value** Head - The "**Critic**"
   - **Purpose**: Estimate how good the current position is
   - **Output**: Scalar value between `-1` (terrible) and `+1` (excellent)
   - **Training**: Mean squared error against actual returns

Self-Play Training

1. Game Generation
2. Training Data Creation
3. Network Training

- The training loop keeps improving the agent through playing the Blackjack
  - Early iterations: Learn basic concepts (don’t hit on 20)
  - Middle iterations: Learn nuanced strategies (16 vs dealer’s 7)
  - Late iterations: Master complex situations (soft hand strategy)
- Each iteration, the agent:
  - Plays against its previous version
  - Collects data where it can improve
  - Updates to beat its previous self
  - The cycle continues until convergence

After playing 10000 games, the win rate stabilizes around 38% - 40%

```
=== Corrected AlphaBlackjack Training ===
Training Corrected AlphaBlackjack...
Episode 0: Win Rate: 0.000, Avg Reward: -1.000, Policy Loss: 0.0000, Value Loss: 0.0000
Episode 1: Win Rate: 0.000, Avg Reward: -1.000, Policy Loss: 0.0000, Value Loss: 0.0000
Episode 2: Win Rate: 0.000, Avg Reward: -1.000, Policy Loss: 0.0000, Value Loss: 0.0000
Episode 3: Win Rate: 0.250, Avg Reward: -0.500, Policy Loss: 0.0000, Value Loss: 0.0000
Episode 4: Win Rate: 0.200, Avg Reward: -0.600, Policy Loss: 0.0000, Value Loss: 0.0000
Episode 5: Win Rate: 0.167, Avg Reward: -0.667, Policy Loss: 0.0000, Value Loss: 0.0000
Episode 6: Win Rate: 0.143, Avg Reward: -0.714, Policy Loss: 0.0000, Value Loss: 0.0000
Episode 7: Win Rate: 0.125, Avg Reward: -0.750, Policy Loss: 0.0000, Value Loss: 0.0000
Episode 8: Win Rate: 0.111, Avg Reward: -0.778, Policy Loss: 0.0000, Value Loss: 0.0000
Episode 9: Win Rate: 0.100, Avg Reward: -0.800, Policy Loss: 0.0000, Value Loss: 0.0000
Episode 500: Win Rate: 0.460, Avg Reward: -0.040, Policy Loss: -195.8588, Value Loss: 0.8951
Episode 1000: Win Rate: 0.280, Avg Reward: -0.370, Policy Loss: -544.8674, Value Loss:
0.8128
Episode 1500: Win Rate: 0.380, Avg Reward: -0.230, Policy Loss: -5339.5576, Value Loss:
2.1377
Episode 2000: Win Rate: 0.460, Avg Reward: -0.060, Policy Loss: -0.0000, Value Loss: 1.0694
Episode 2500: Win Rate: 0.330, Avg Reward: -0.270, Policy Loss: -974.2815, Value Loss:
0.9455
```

After playing 10000 games, the win rate stabilizes around 38% - 40%

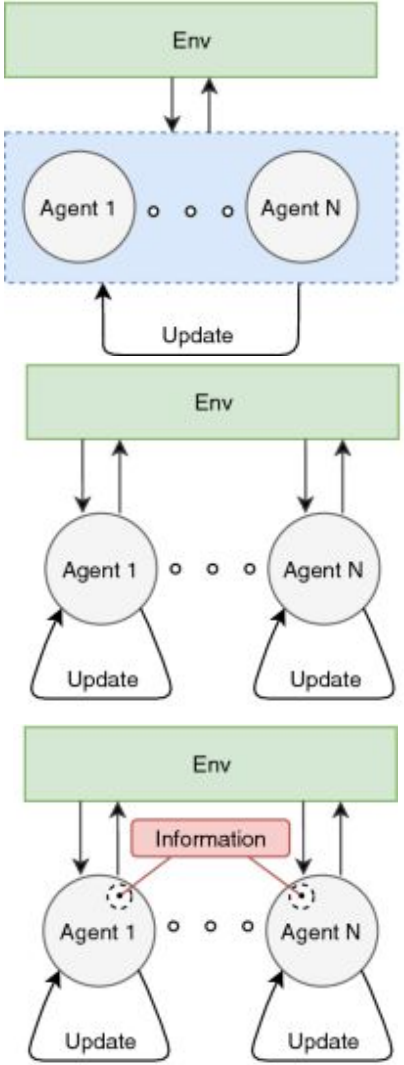
```
Evaluating over 10000 episodes...
Results: 3860 wins, 5647 losses, 493 draws
Win rate: 0.386
Loss rate: 0.565
Draw rate: 0.049
Average reward: -0.179

=== Policy Analysis ===
State -> Action probabilities (STAND, HIT):
(20, 5, False) -> STAND ✓ (stand: 1.000, hit: 0.000, value: -0.953)
(11, 2, False) -> STAND ✗ (stand: 1.000, hit: 0.000, value: -0.754)
(16, 7, False) -> STAND ✗ (stand: 1.000, hit: 0.000, value: -1.018)
(16, 6, False) -> STAND ✓ (stand: 1.000, hit: 0.000, value: -0.972)
(12, 2, False) -> STAND ✓ (stand: 1.000, hit: 0.000, value: -0.761)
(12, 9, False) -> STAND ✗ (stand: 1.000, hit: 0.000, value: -1.082)
(18, 9, False) -> STAND ✓ (stand: 1.000, hit: 0.000, value: -1.123)
(8, 6, False) -> STAND ✗ (stand: 1.000, hit: 0.000, value: -0.917)
(13, 10, False) -> STAND ✗ (stand: 1.000, hit: 0.000, value: -1.135)
(17, 10, False) -> STAND ✓ (stand: 1.000, hit: 0.000, value: -1.162)

Model saved! Final win rate: 0.386
```

A Single Agent for Multiple Games

Image credit: Springer
Code:
https://colab.research.google.com/drive/110Zfh5LSQAZwADFQBKvr2rWbKhwd6T6t?usp=sharing



Challenges for A Single Agent for Multiple Games

- Current Standard: Most RL agents are specialists (e.g. AlphaGo for Go, Libratus for Poker).
- Problem: **Catastrophic Forgetting**
  - When a neural network learns a new task (Game B), it overwrites the weights that were crucial for the previous task (Game A), causing it to **forget** how to play Game A.

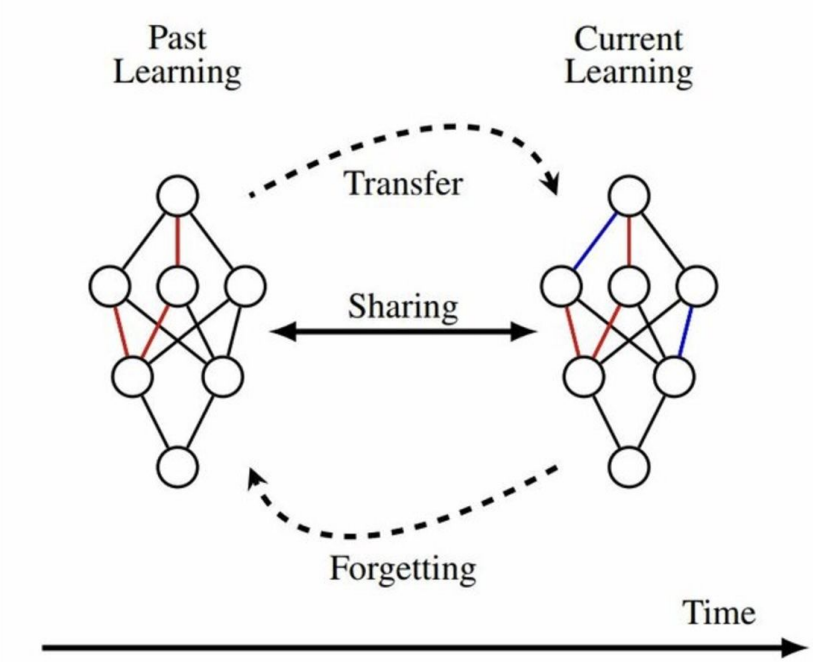
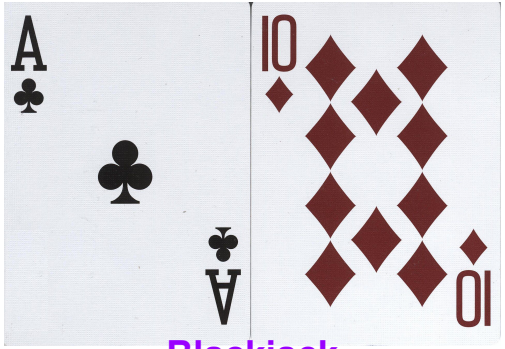


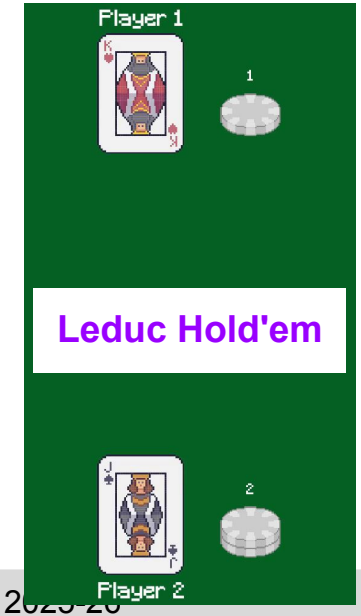
Image credit: UnfoldAI

Target Games

1. **Blackjack**: Simple, perfect information (with fixed deck), focused on counting.
2. **Leduc Hold'em**: A simplified Poker with a 6-card deck. Introduces imperfect information and bluffing.
3. **Hearts**: A trick-taking game with complex, long-term strategy and partnerships.



Blackjack



Hearts

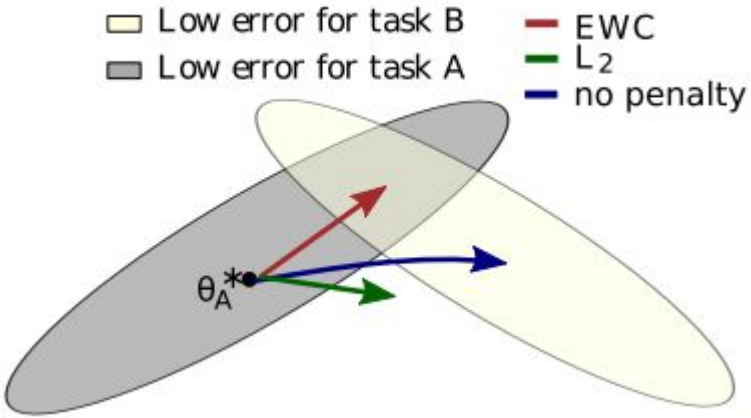
Deep Q-Network (DQN) Architecture

1. **State Representation**: A vector encoding the game state (e.g., player's hand, cards on table, current turn, history of actions, … etc).
2. **Action Space**: The set of legal moves (e.g., Hit/Stand, Bet/Fold, which card to play).
3. **Reward Signal**: +1 for win, -1 for loss, 0 otherwise (or intermediate rewards for tricks won, etc.).
4. **Neural Network**: A multi-layer perceptron (MLP) that takes the state and outputs Q-values for each action.

Elastic Weight Consolidation

- Solution to Catastrophic Forgetting
- **Synaptic consolidation**: Some synapses in the brain are more important for retaining old memories and are hardened against change.
- EWC penalizes large changes in the parameters that are important for previous tasks while allowing more flexibility in parameters important for the current task.

$$\mathcal{L}(\theta) = \mathcal{L}_B(\theta) + \sum_i \frac{\lambda}{2} F_i (\theta_i - \theta^*_{A,i})^2$$



Reference: https://arxiv.org/pdf/1612.00796

Applying EWC to Card Game Agent

1. Train on Game A (e.g., Poker)
   - Train the DQN agent until convergence.
2. Train on Game B (e.g., Blackjack) with EWC
   - Initialize the network with the parameters from Game A.
   - This allows the network to learn Blackjack but is “elasticated” - it can change unimportant weights freely but is pulled strongly back towards the good Poker solution for important weights.
3. Repeat for Game C (e.g., Hearts)
   - Now the EWC loss would contain terms for both Poker and Blackjack

- We need to teach the agent how to use the spare neurons step-by-step.

Sample run

```
Using device: cpu
=== Training DQN on Blackjack + Leduc + Hearts ===
Training started...
Ep    0 | Reward: -1.00 | Avg: -1.00 | ε: 0.995 | Game: leduc
Ep  200 | Reward: -1.00 | Avg: -2.58 | ε: 0.365 | Game: blackjack
Ep  400 | Reward: -10.00 | Avg: -2.17 | ε: 0.134 | Game: leduc
Ep  600 | Reward: -2.00 | Avg: -2.32 | ε: 0.050 | Game: hearts
Ep  800 | Reward: -5.00 | Avg: -1.70 | ε: 0.050 | Game: hearts
Ep 1000 | Reward: -1.00 | Avg: -2.60 | ε: 0.050 | Game: blackjack
Ep 1200 | Reward: -23.00 | Avg: -2.80 | ε: 0.050 | Game: hearts
Ep 1400 | Reward:  1.00 | Avg: -0.99 | ε: 0.050 | Game: blackjack

=== Evaluation ===
blackjack : -0.526 (2000 eps)
leduc     : 0.637 (1000 eps)
hearts    : -6.180 (50 eps)

Model saved as 'dqn_multi_game.pth'
Ready for benchmarks or deployment!
```

TED^{X}Exeter Talk

https://www.youtube.com/watch?v=mqma6GpM7vM

Artificial Intelligence, Video Games and the Mysteries of the Mind (Raia Hadsell)

Reference: 1 agent can learn to play 10 different arcade games without forgetting.

Self-Study

3D Model Generation

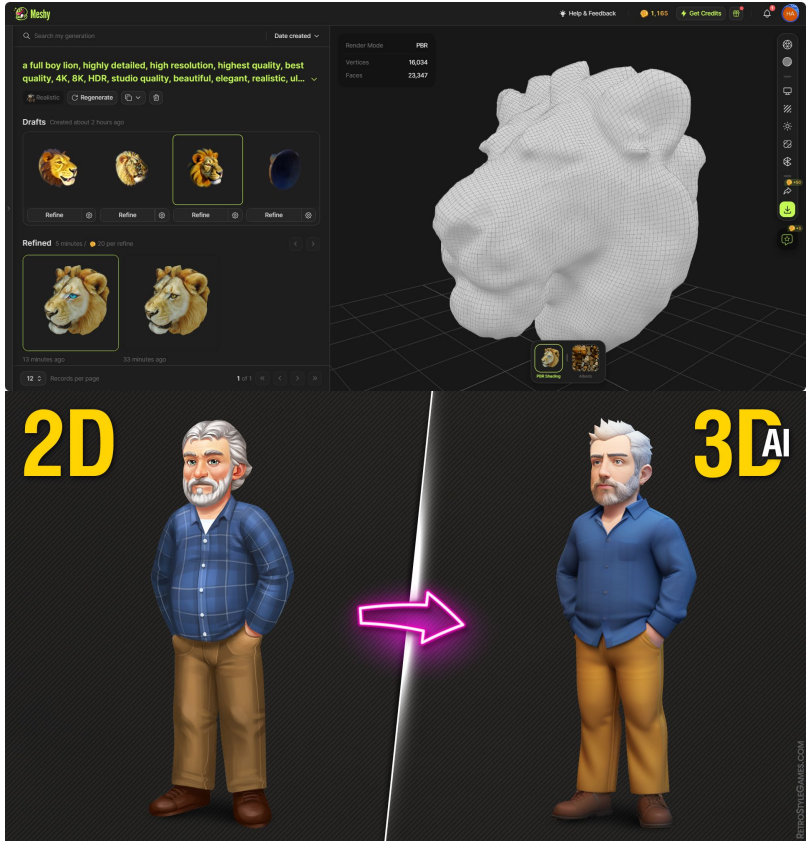


Image credit: Google Image

Use Cases for 3D Model Generation

1. **Game Development**: Rapid prototyping of assets.
    - Accelerated ideation, democratization of creation, cost reduction
2. **Architects and Designers**: Quick visualization of concepts.
    - Creative exploration, instant client proposals, functional prototyping
3. **Marketers**: Creating 3D product visuals for advertisements.
    - Personalized marketing, enhanced AR experience
4. **Everyone**: Lowering the barrier to 3D creation.
    - Visualizing the imagination, streamlined workflows, cost reduction
5. … etc

Why is 3D Content Creation So Hard?

1. **The Skills Bottleneck**: Requires mastering multiple complex disciplines, including modeling, texturing, rigging and lighting, within non-intuitive software.
2. **The Time Sink**: Manipulating millions of polygons is inherently slow, and even minor changes can trigger hours of tedious rework.
3. **The Technical Gauntlet**: Demands precise and lots of geometry computations with perfect topology.
4. **The Artistic Burden**: Every step requires deep artistic judgment for form, proportion and material authenticity.

3D Shape Representation

1. **Mesh**: A collection of vertices, edges, and faces that define the shape of a 3D object. (We cover this one in this course)
2. **Depth Map**: A 2D image where each pixel value represents the distance from a camera viewpoint to the corresponding object in the 3D scene, rather than its color.
3. **Voxel Grid**: A 3D boolean array representing the object.
4. **Pointcloud**: A set of data points in a 3D coordinate system (X, Y, Z).
5. **Implicit Surface**: A 3D shape as the set of points that satisfy a given function $f(x, y, z) = 0$ (or ≤ 0).

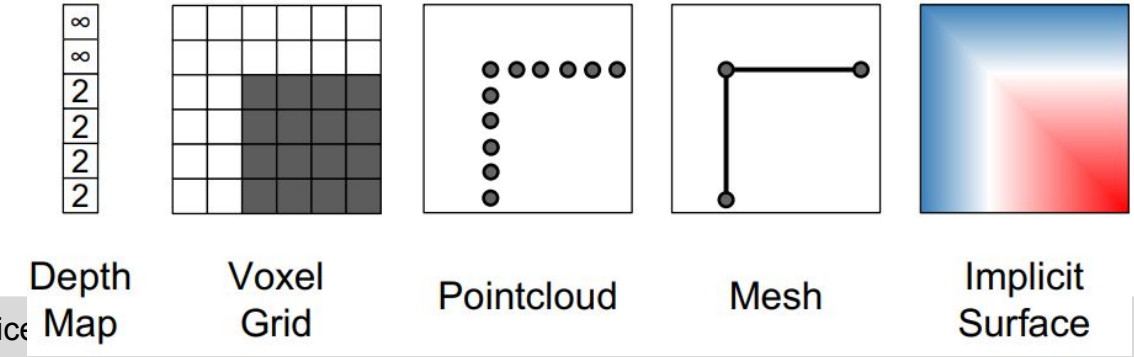
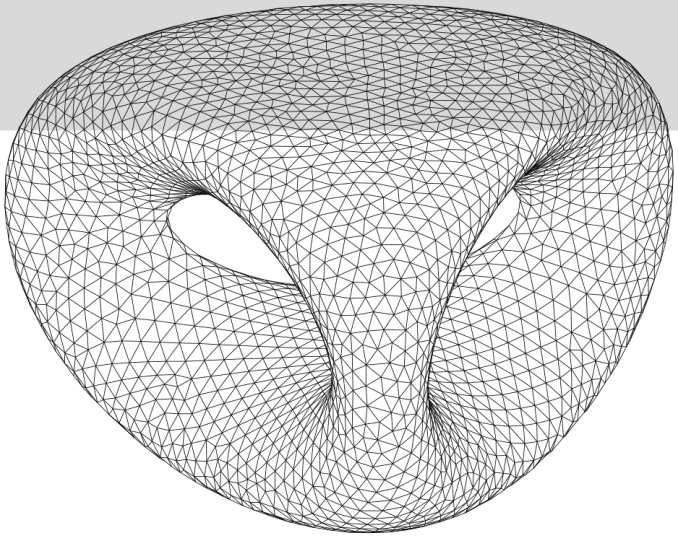


Image credit: https://cs231n.stanford.edu/2024/slides/2024/lecture_18.pdf

Triangle Mesh

- Represent a 3D shape as a set of triangles
- **Vertices**: Set of V points in 3D space
- **Faces**: Set of triangles over the vertices
- (+) Standard representation for graphics
- (+) Explicitly represents 3D shapes
- (+) Adaptive: Can represent flat surfaces very efficiently, can allocate more faces to areas with fine detail
- (+) Can attach data on verts and interpolate over the whole surface: RGB colors, texture coordinates, normal vectors, etc.
- (-) Non-trivial to process with neural networks



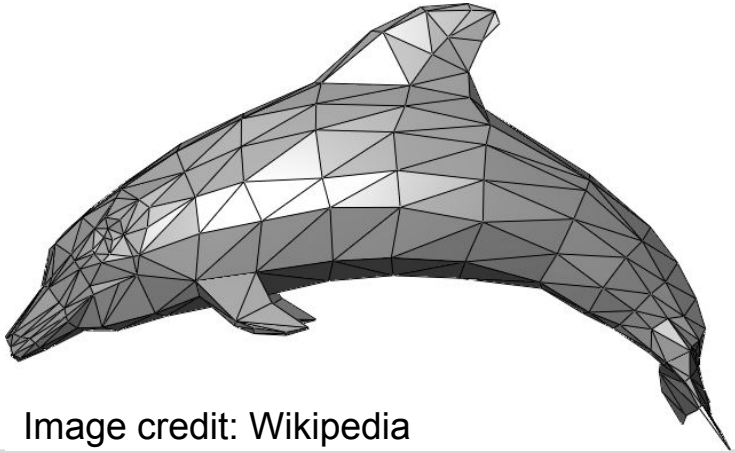


Image credit: Wikipedia

Example: Triangle Mesh

```python
import trimesh
import numpy as np
# Define a simple cube as a mesh, 8 vertices of a cube
vertices = np.array([
    [0, 0, 0], [1, 0, 0], [1, 1, 0], [0, 1, 0],  # Bottom face
    [0, 0, 1], [1, 0, 1], [1, 1, 1], [0, 1, 1]   # Top face
])
# 12 triangles (2 per face of the cube)
faces = np.array([
    [0, 1, 2], [0, 2, 3],  # Bottom
    [4, 5, 6], [4, 6, 7],  # Top
    [0, 4, 7], [0, 7, 3],  # Left
    [1, 5, 6], [1, 6, 2],  # Right
    [0, 1, 5], [0, 5, 4],  # Front
    [3, 2, 6], [3, 6, 7]   # Back
])
mesh = trimesh.Trimesh(vertices=vertices, faces=faces)  # Create a mesh object
print(f"Mesh: {len(vertices)} vertices, {len(faces)} faces")
mesh.show() # visualize the mesh
```

Why are Meshes popular among the choices?

1. Meshes provide continuous surfaces, without holes
2. Meshes use less memory for surfaces
3. Meshes are directly renderable
4. Meshes represent full 3D geometry

- That’s why there are more tools available for meshes

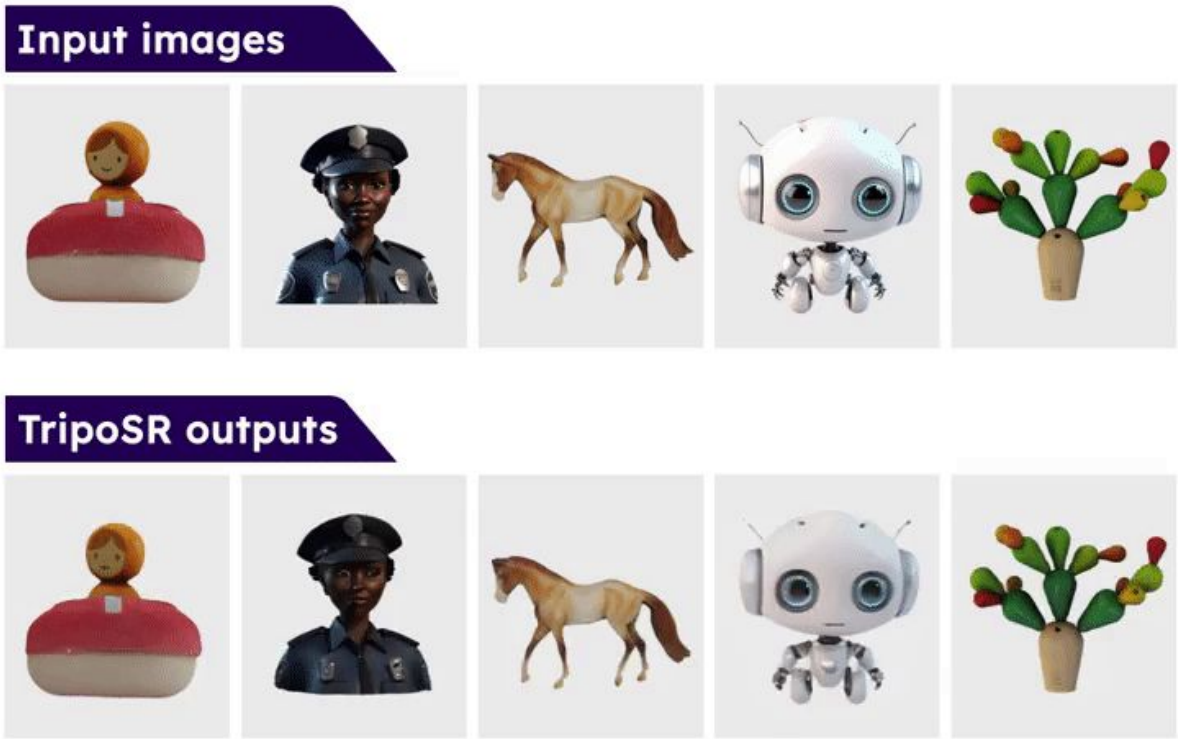
- In terms of deep learning,

1. Meshes fit better with Iterative refinement on the surface
2. Meshes provides aligned vertex features
3. Meshes can be converted to pointclouds directly, then compute loss

2D-to-3D: TripoSR

Publication: https://arxiv.org/abs/2403.02151
Usage: https://github.com/VAST-AI-Research/TripoSR

Samples from TripoSR



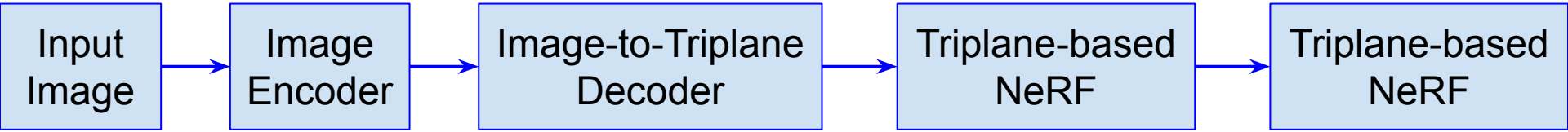
Single-Image 3D Reconstruction

- Task: To predict a full 3D shape from a single RGB image
  - Input: Image
  - Output: 3D mesh for the object in the image

- Major Challenge: infinite 3D shapes can explain a single 2D view.
- Requires strong priors and learning from vast amounts of data.

Multi-stage Process in TripoSR

- Does not require camera parameters as input.
- Learns to guess them during training, making it robust for images.



1. **Input**: Single RGB Image
2. **Image Encoder**: (DINOv1) extracts features.
3. **Image-to-Triplane Decoder**: (Transformer) maps image features to a 3D representation.
4. **Triplane-based NeRF**: Generates color and density for any 3D point.
5. **Output**: 3D Mesh (via Marching Cubes).

Improvement on Data

- **Data Curation**:
  - Used a carefully curated, high-quality subset of the Objaverse dataset (CC-BY licensed).

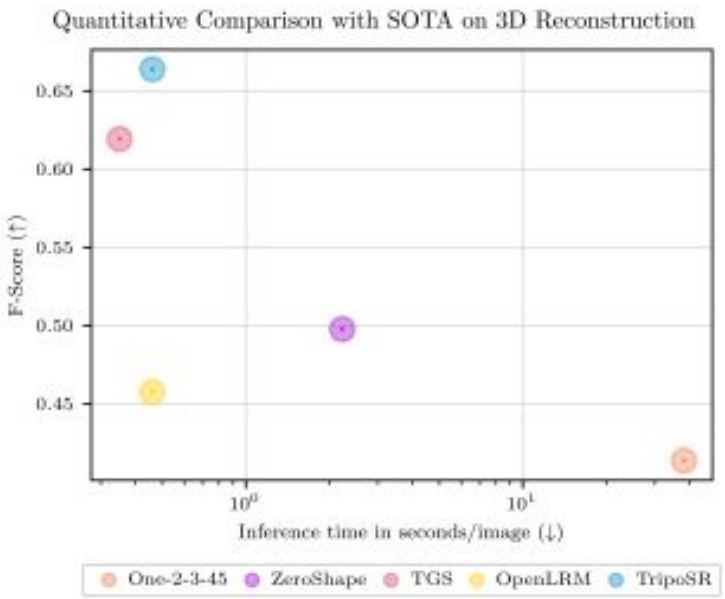
- **Data Rendering**:
  - Employed diverse rendering techniques to mimic the distribution of real-world photos.
  - This improves generalization even when trained on synthetic data.

Improvement on Model and Training

- **Triplane Channel Optimization**:
  - Experimentally chose 40 channels for the triplane.
  - Balances detail (high fidelity) and efficiency (memory/ speed).
- **Mask Loss**:
  - Added a Binary Cross-Entropy (BCE) loss on rendered silhouettes.
  - Significantly reduces "floater" artifacts and improves reconstruction fidelity.
- **Local Rendering Supervision**:
  - Instead of rendering full 512x512 images, render random 128x128 patches.
  - Uses importance sampling to focus on foreground objects.
  - Enables learning fine details without overwhelming GPU memory.

Evaluation for TripoSR

- Evaluation Datasets:
  - **GSO (Google Scanned Objects)** and **OmniObject3D**.
  - Manually filtered to ensure a diverse, non-trivial test set.
- Metrics:
  - **Chamfer Distance (CD)**: Measures geometric accuracy (lower is better).
  - **F-Score (FS)**: Measures how well the surfaces match at a given distance threshold (higher is better).
- Baselines: Compared against One-2-3-45, ZeroShape, TGS, and OpenLRM.



Evaluation for TripoSR

Method	CD↓	FS@0.1↑	FS@0.2↑	FS@0.5↑
One-2-3-45 [16]	0.227	0.382	0.630	0.878
ZeroShape [13]	0.160	0.489	0.757	0.952
TGS [35]	0.122	0.637	0.846	0.968
OpenLRM [10]	0.180	0.430	0.698	0.938
TripoSR (ours)	**0.111**	**0.651**	**0.871**	**0.980**

Table 2. Quantitative comparison of different techniques on GSO [6] validation set, where CD and FS refer to Chamfer Distance and F-score respectively.

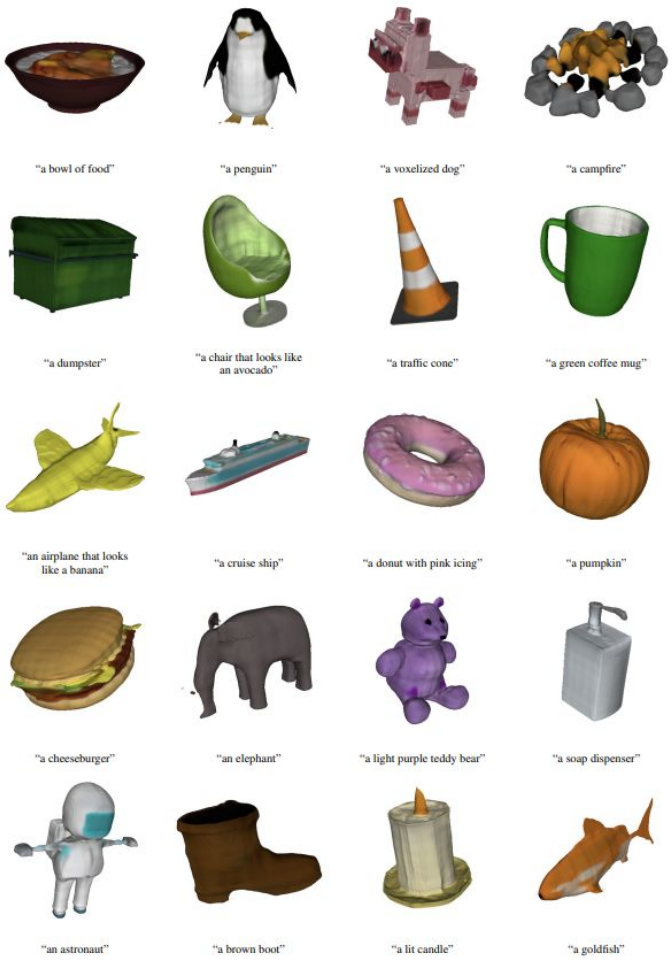
Method	CD↓	FS@0.1↑	FS@0.2↑	FS@0.5↑
One-2-3-45 [16]	0.197	0.445	0.698	0.907
ZeroShape [13]	0.144	0.507	0.786	0.968
TGS [35]	0.142	0.602	0.818	0.949
OpenLRM [10]	0.155	0.486	0.759	0.959
TripoSR (ours)	**0.102**	**0.677**	**0.890**	**0.986**

Table 3. Quantitative comparison of different techniques on OmniObject3D [30] validation set, where CD and FS refers to Chamfer Distance and F-score respectively.

Other Approaches: 2D-to-3D

1. MiDaS for Depth Estimation and 3D Reconstruction
2. PyTorch3D with Differentiable Rendering
3. Open3D’s Built-in Reconstruction

Text-to-3D: Shap-E



Publication: https://arxiv.org/abs/2305.02463
Usage: https://github.com/openai/shap-e?tab=readme-ov-file

Samples from Shap-E

A chair that looks like an avocado

An airplane that looks like a banana

A spaceship

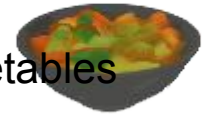
A birthday cupcake

A chair that looks like a tree

A green boot

A penguin

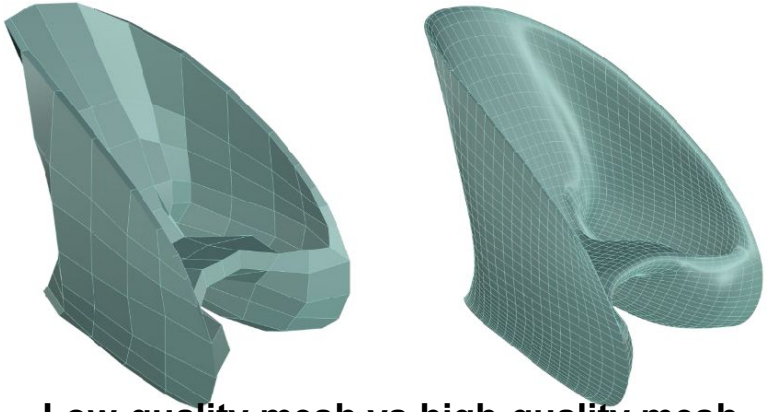
Ube ice cream cone



A bowl of vegetables

Why Generate 3D Assets with Implicit Functions?

- Traditional 3D reps (e.g., meshes, point clouds) are resolution-limited and inflexible for tasks like rendering or editing.
- INRs (e.g., NeRFs) enable resolution-independent, differentiable 3D modeling – ideal for novel views, style transfer, and editing.
- Problem: Acquiring INRs is computationally expensive (per-asset optimization takes hours); modeling their high-dimensional parameters is challenging.
- Impact: Enables fast, scalable 3D creation for AR/VR, gaming, and design – democratizing high-quality 3D content without manual effort.



Low-quality mesh vs high-quality mesh

Image credit: ResearchGate

Implicit Neural Representation (INR)

- An Implicit Neural Representation is the data representation using neural network. Instead of storing the data explicitly, it learns a continuous function that can answer queries about the data.
- In data representation III, we covered INR using autoencoder (latent space). We also covered INR using transformer (token embeddings + position encoding)

Shap-E – Direct Generation of 3D Implicit Functions

- Main contribution: First model to directly sample INR parameters via diffusion, bypassing slow per-sample optimization – produces complex assets in ~13 seconds.
- A two-stage conditional generative model that outputs parameters of INRs representing 3D assets.
- Supports text-to-3D and image-to-3D conditioning for diverse outputs (e.g., "a red sports car" or input sketch).
- Generates assets as both textured meshes (via Signed Distance Functions) and NeRFs for photorealistic rendering.

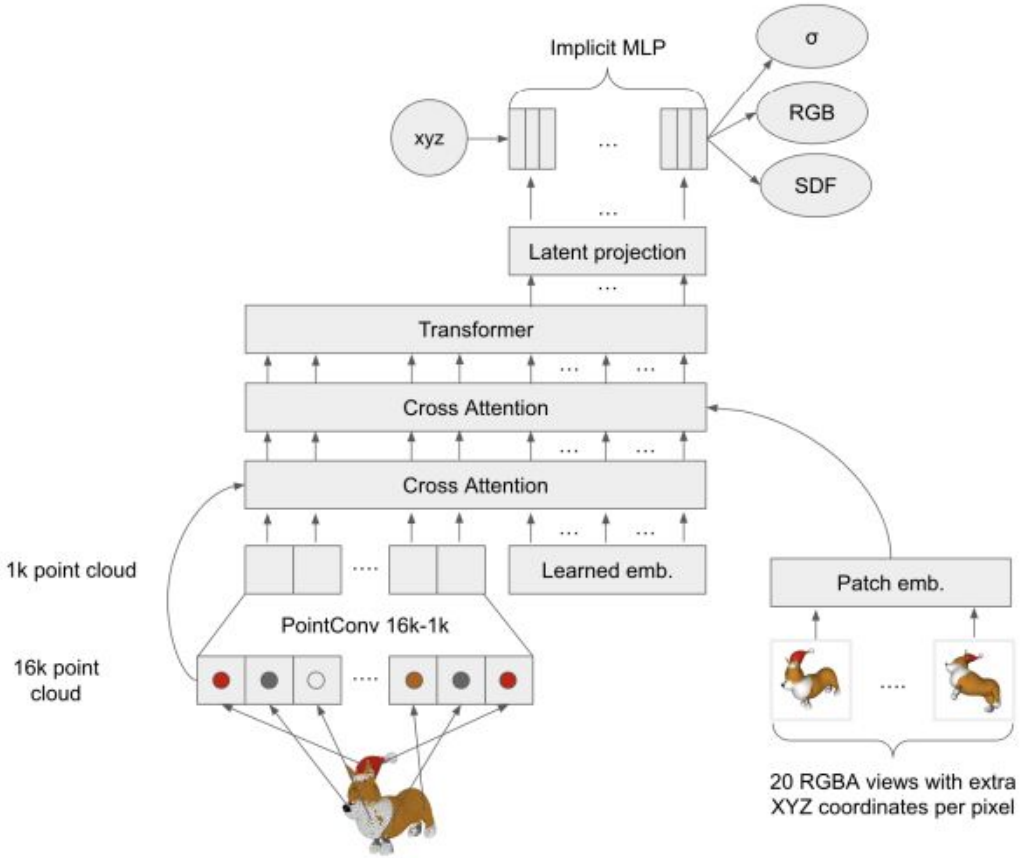
Shap-E: Encoder Architecture

Transformer processes point clouds

+ multiview renders

→ Outputs compact INR parameters

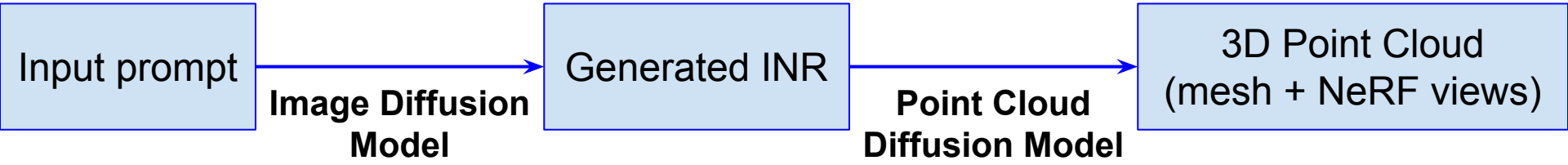
(MLP weights for NeRF or SDF)



Two-Stage Training and Generation Pipeline

1. **Generate a Synthetic View**: Use a text-conditional image diffusion model to create a 2D rendering of the object.
2. **Generate a 3D Point Cloud**: Use an image-conditional point cloud diffusion model to produce the 3D geometry and color from the generated image.

- Generation and Rendering:



- Shap-E is trained on millions of 3D-text/image pairs with zero-shot capability

A Diffusion Model for 3D Points

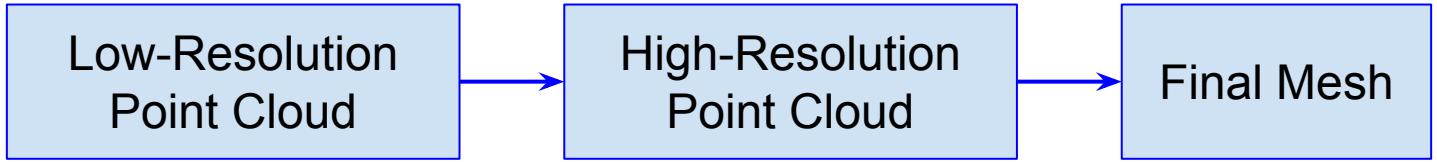
- How do you apply diffusion (designed for images) to an unordered set of points? Shapt-E's solution:

1. Represents a point cloud as a set of **3D coordinates** (x,y,z) and **RGB colors**.
2. **Forward Process**: Adds Gaussian noise to both the coordinates and colors of the points.
3. **Reverse Process**: A neural network learns to **denoise** this noisy point cloud, gradually recovering the clean 3D shape.
4. **Conditioning**: The model is conditioned on the generated image from the first step via a **CLIP image embedding**.

Point Clouds are Sparse. How can we get a mesh?

- Shap-E's Solution: An Upsampler

1. A second, **super-resolution diffusion model** takes the low-res point cloud and generates a **high-resolution point cloud** (4,096 points).
2. Standard algorithms (like **Screened Poisson Surface Reconstruction**) can then convert this dense point cloud into a smooth, watertight mesh.



How It Performs: Speed, Quality and Flexibility

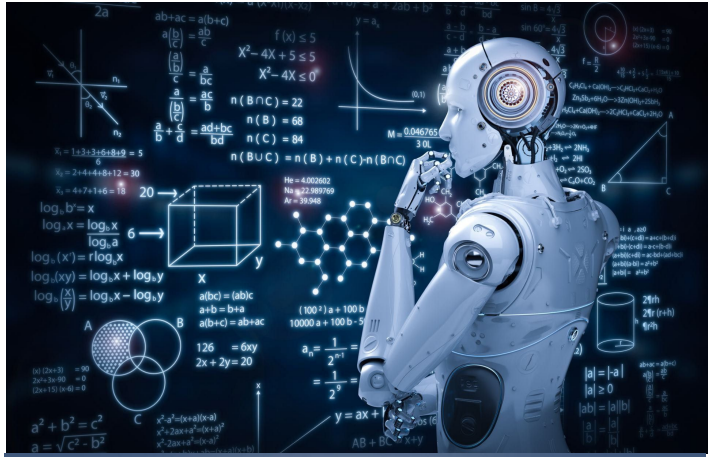
1. **Speed**: ~13s per asset on V100 GPU (vs. hours for optimization baselines).
2. **Quality**: measured by FID and CLIP score, and significantly better than the work before 2023.
3. **Flexibility**: Handles complex prompts (e.g., “donut with pink icing”); outputs editable INRs for downstream tasks.
4. **Limitations** for shap-E:
   - **Lower Fidelity**: Outputs are less detailed and coherent than slower, SOTA methods.
   - **Point Cloud Artifacts**: Produce “floaters” (disconnected points) and noisy geometry.
   - **Topological Errors**: Struggles with complex structures and precise object composition.
   - **Dependence on 2D Model**: The quality of the final 3D model is bottlenecked by the quality of the first-stage 2D image generator.

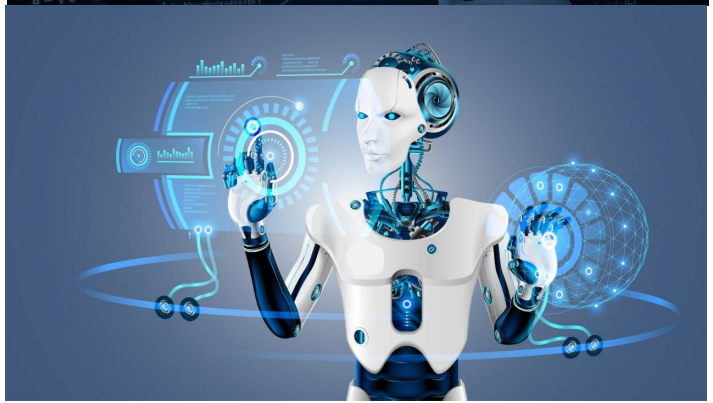
Why Shap-E Matters: Unlocking 3D Creativity

- Accelerates 3D content creation 100x+ faster than prior methods, fostering innovation in media, simulation, and AI design.
- Open-source models/code released – enables community extensions (e.g., better multi-object support).
- Potential improvements on Shap-E:

1. Address biases
2. Improve detail resolution
3. Integrate with real-time editing tools.
4. Animate the generated objects

Robot Learning

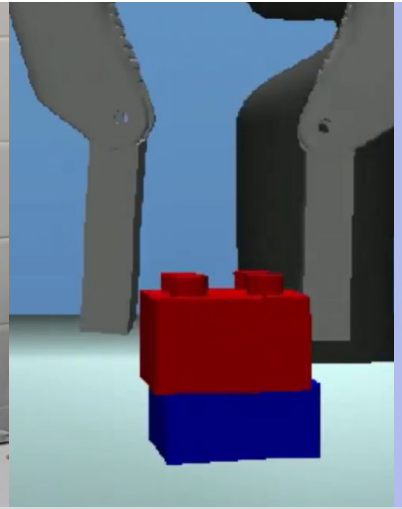




Reference: https://cs231n.stanford.edu/slides/2025/lecture_17.pdf
Image credit: Google Image

Online Robot Learning Demonstrations

1. Learning to walk: https://www.youtube.com/watch?v=n2gE7n11h1Y
2. Learning to walk and run: https://www.youtube.com/@aiwarehouse
3. Learning to grasp: https://www.youtube.com/watch?v=noEGSjQMi7Q
4. Learning to manipulate: https://www.youtube.com/watch?v=8QnD8ZM0YCo
5. Learning to dress: https://www.youtube.com/watch?v=ixmE5nt2o88



A Fast-Growing Field

Toyota Research Institute

Meta AI Research

Google Robotics

Nvidia Research

Image credit: https://cs231n.stanford.edu/slides/2025/lecture_17.pdf

Robot Perception

- Incomplete knowledge of objects and scenes
- Imperfect actions may lead to failure
- Environment dynamics and other agents
- Understanding the interactions with the world through multimodal senses

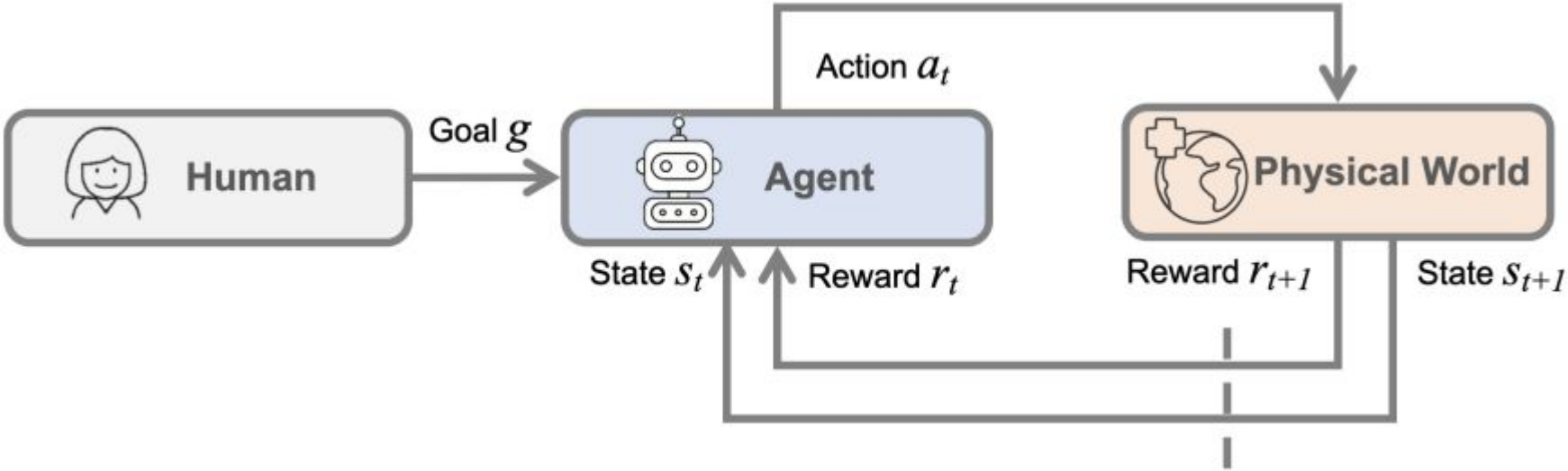
Robot Vision

- Robot vision is **embodied**, **active** and environmentally **situated**.

1. **Embodied**: Robots have physical bodies and experience the world directly. Their actions are part of a dynamic with the world and have immediate feedback on their own sensation.
2. **Active**: Robots are active perceivers. It knows why it wishes to sense, and chooses what to perceive, and determines how, when and where to achieve that perception.
3. **Situated**: Robots are situated in the world. They do not deal with abstract descriptions, but with the “here” and “now” of the world directly influencing the behavior of the system.

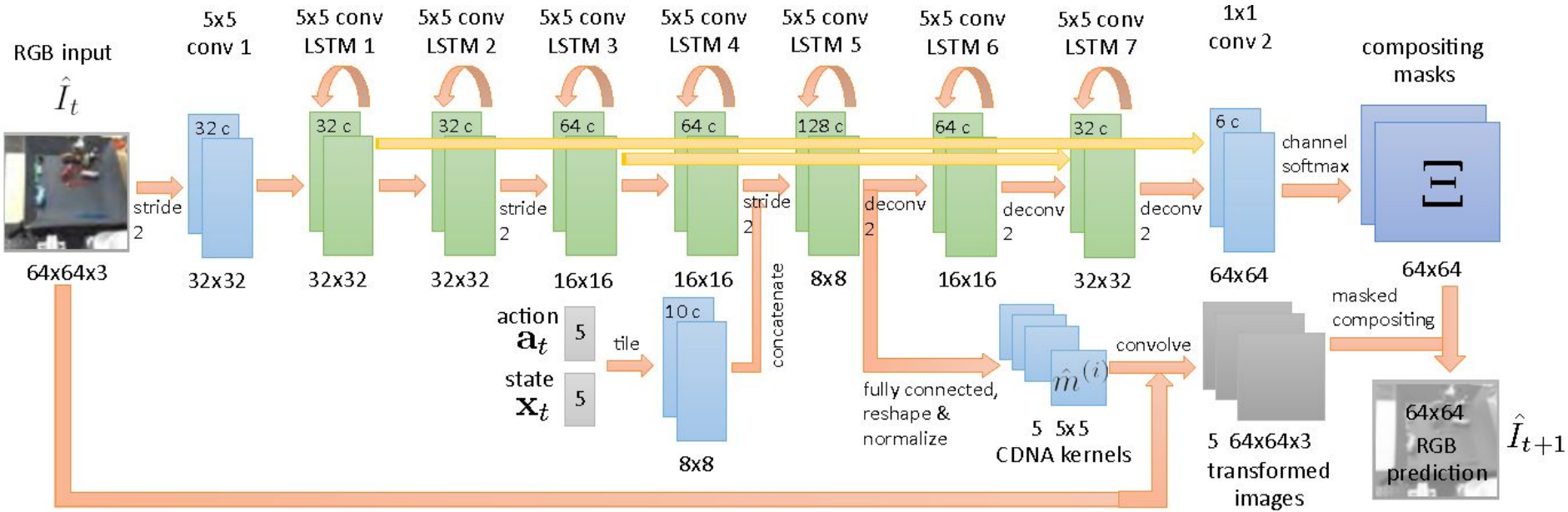
Robot Learning is to construct the perception-action loop.

Formulating Robot Learning as Reinforcement Learning



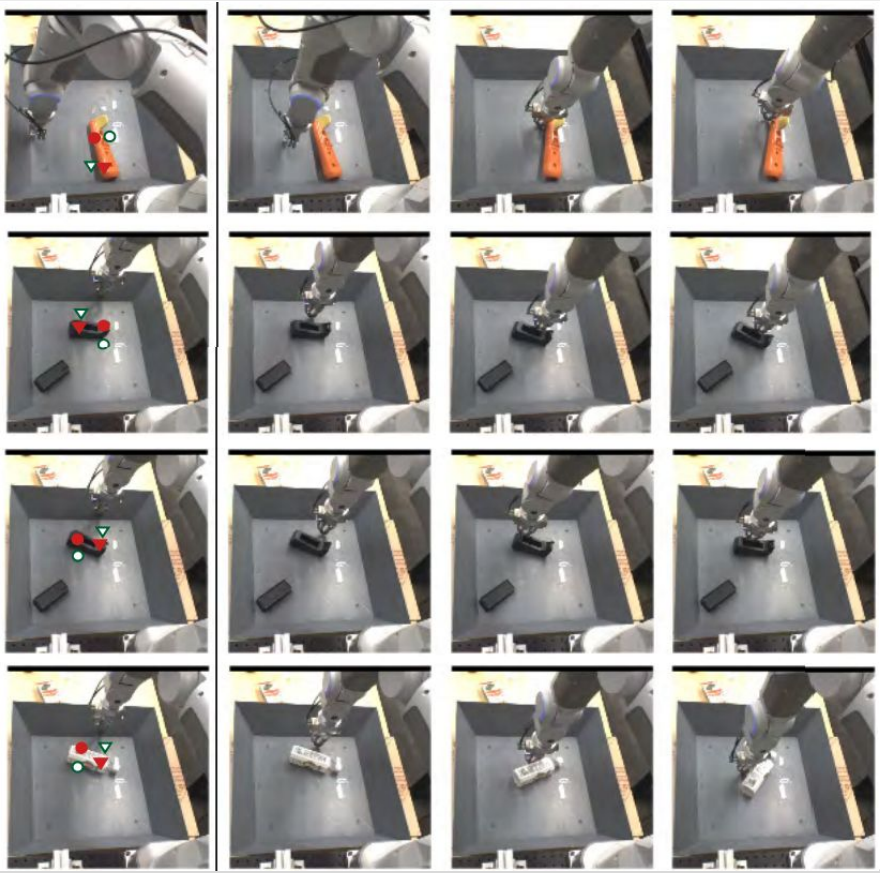
Pixel Dynamics: Deep Visual Foresight

- Learn a model of the world's state transition function $P(s_{t+1}|s_t, a_t)$ and then use planning through the model to make decisions

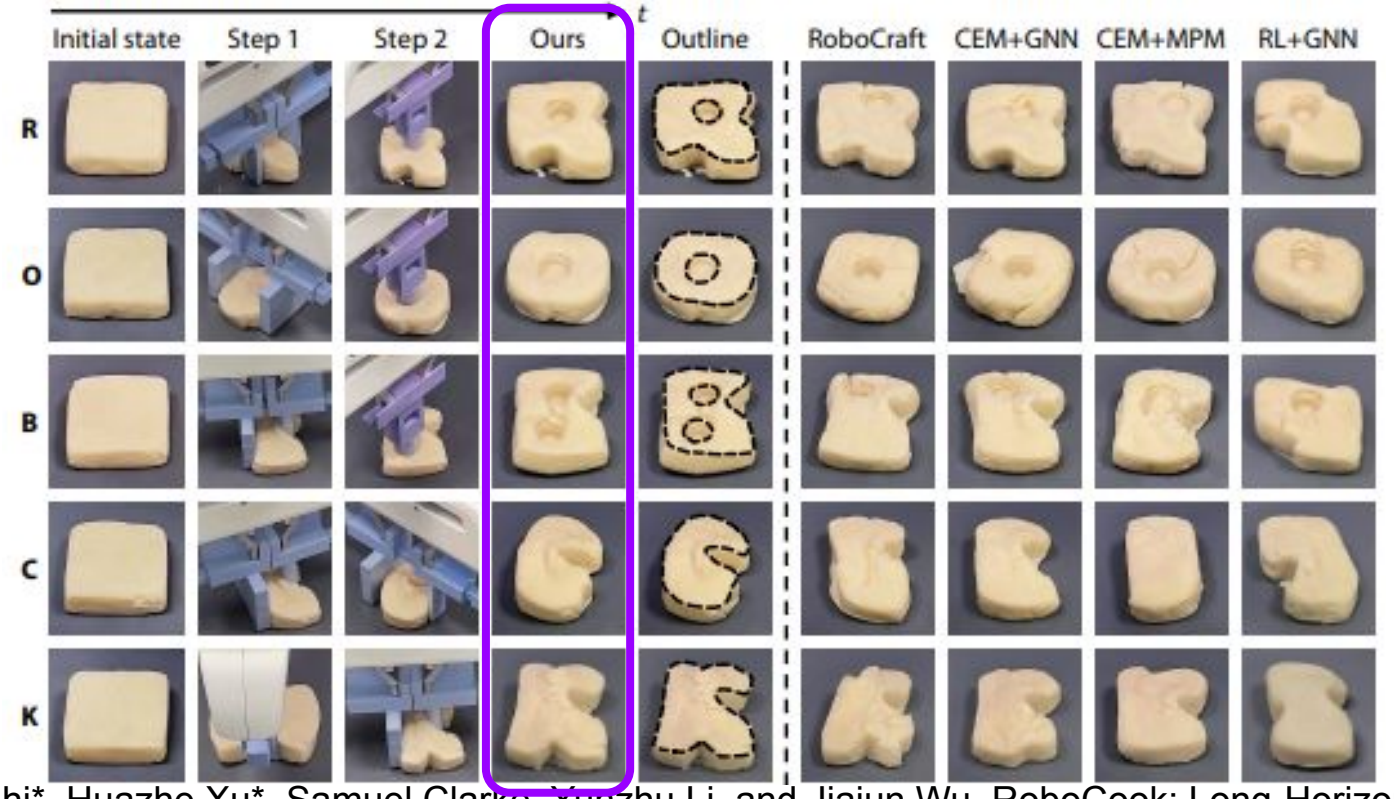


Reference: Finn and Levine, "Deep Visual Foresight for Planning Robot Motion", ICRA 2017

In 2017, Robot arm is able to rotate the objects

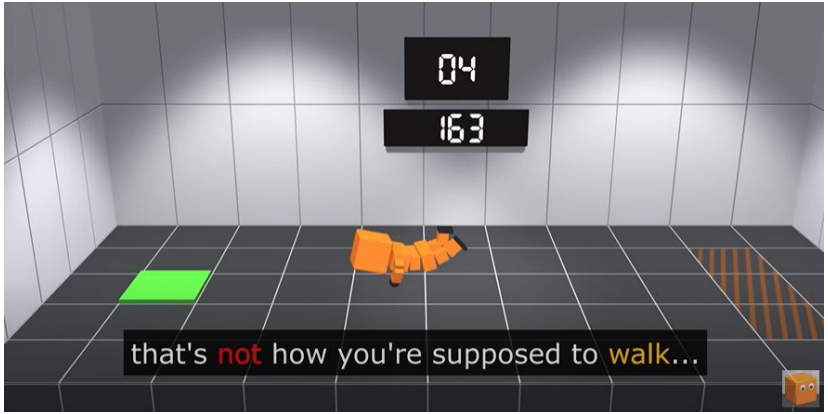


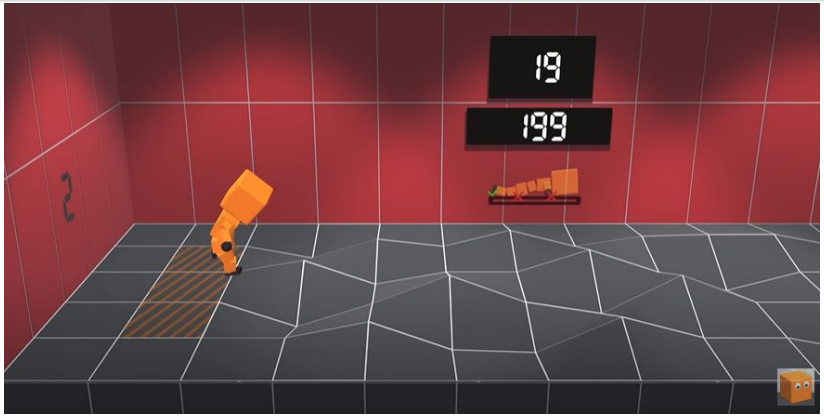
In 2023, Robot arm is able to make alphabet cookies

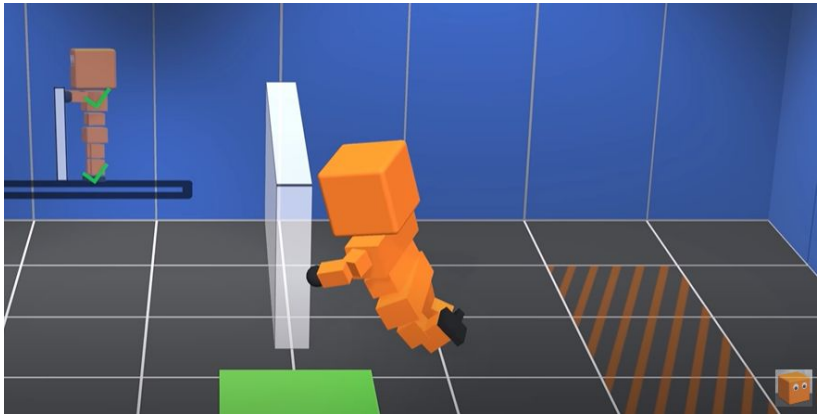


Haochen Shi*, Huazhe Xu*, Samuel Clarke, Yunzhu Li, and Jiajun Wu. RoboCook: Long-Horizon Elasto-Plastic Object Manipulation with Diverse Tools. Conference on Robot Learning (CoRL) 2023

Case Study: Learn to Walk







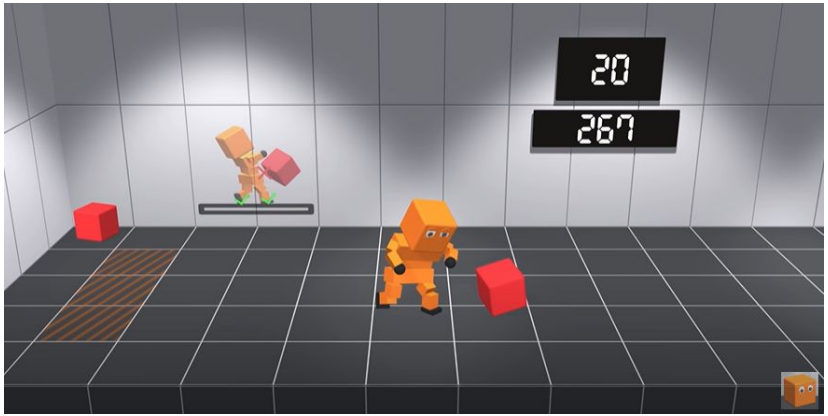


Image credit: https://www.youtube.com/watch?v=L_4BPjLBF4E&t=157s&ab_channel=AIWarehouse

Transfer Learning

- In practice, very few people train an entire deep learning model from scratch (with random initialization), because it is relatively **rare to have a dataset of sufficient size**. Instead, it is common to pretrain a DL model on a very large dataset, and then use the DL model either as an initialization or a fixed feature extractor for the task of interest.
- There are **three major transfer learning scenarios**.

Reference: https://cs231n.github.io/transfer-learning/

Transfer Learning Scenarios

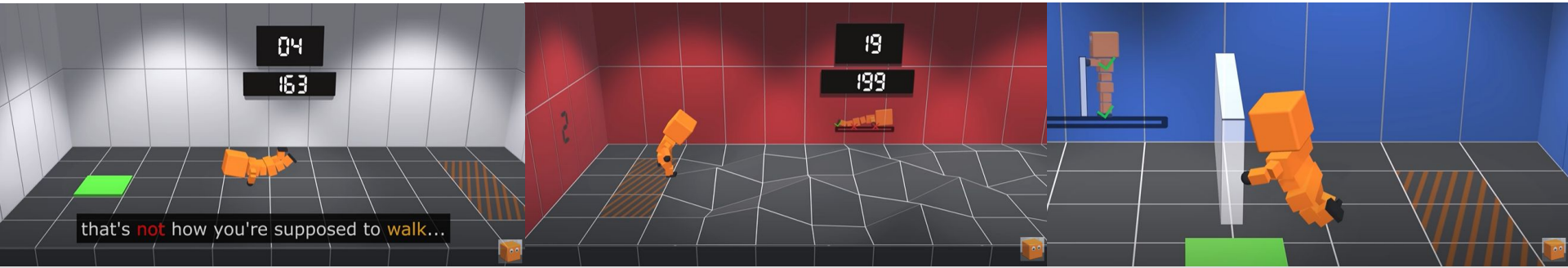
1. **Fixed feature extractor**: Take a pre-trained DL model, and remove the last fully-connected layer, then treat the rest of the DL model as a fixed feature extractor for the new dataset.
2. **Fine-tuning the DL model**: Fine-tune the weights of the pretrained network by continuing the backpropagation.
3. **Pretrained models**: Some libraries provide pre-trained models for various applications, e.g. the Caffe library has a Model Zoo where people share their network weights

When and how to fine-tune?

1. New dataset is **small** and **similar** to original dataset: Train a linear classifier on the fixed feature extractors
2. New dataset is **large** and **similar** to the original dataset: We won't overfit if we were to try to fine-tune through the full network
3. New dataset is **small** but very **different** from the original dataset: Train the SVM classifier from activations somewhere earlier in the network
4. New dataset is **large** and very **different** from the original dataset: We have enough data and confidence to fine-tune through the entire network.

Transfer Learning: Back to Learn to Walk

1. The humanoid learns to crawl across the floor from a source to a sink.
2. Humanoid then learns to walk by using a wall for support.
3. Due to time constraints, humanoid learns to walk independently without the wall's aid.
4. The knowledge gained by the humanoid is then transferred to more complex locomotion tasks, such as walking and running.



Imitation Learning

- A framework where an agent learns to perform a task by observing and mimicking expert demonstrations, rather than learning from reward signals.

- Humanoid Example: The humanoid watches videos of humans walking and tries to replicate the same motion patterns. It doesn't understand why these motions work (like balance or efficiency), it just copies what it sees.

Learning from Demonstration

- Often used synonymously with Imitation Learning, but typically refers more specifically to the scenario where demonstrations are provided directly to the robot, frequently through teleoperation or kinesthetic teaching.

- Humanoid Example: A researcher physically guides the humanoid's legs through the correct walking motion multiple times. The robot records these joint angles and body positions, then tries to reproduce them autonomously.

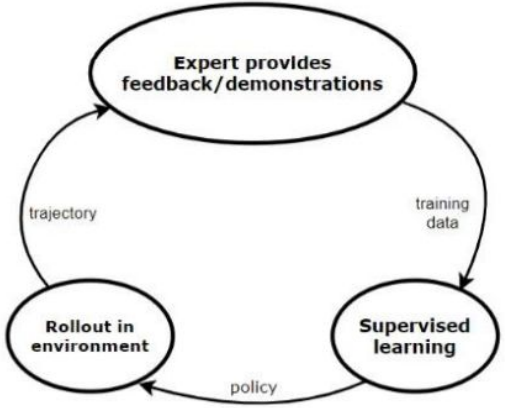
Behaviour Cloning

- A specific, simple technique within Imitation Learning that treats the problem as supervised learning. The agent learns a direct mapping from states/observations to actions.

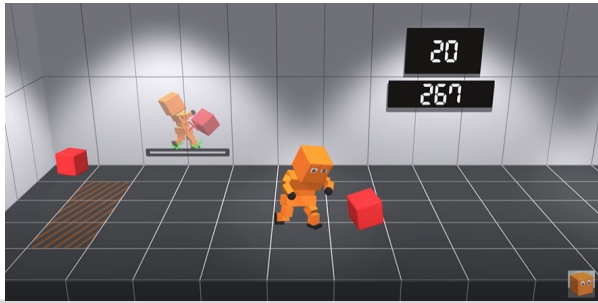
- Humanoid Example: The humanoid receives a dataset of `(body_pose, joint_angles)` pairs from expert walking demonstrations. It then trains a neural network to predict the expert's joint angles given the current body pose.

Iterative Collection of Expert Demonstrations

- An interactive process where the agent practices, identifies its weaknesses, requests new demonstrations for those specific failure cases, and improves iteratively.



- Humanoid Example:
  - Humanoid attempts to walk
  - Expert observes humanoid movement
    - Expert will punish (throws an obstacle) to humanoid if the humanoid movement is not natural, e.g. use left leg to walk consecutively
  - Humanoid learns from feedback



Inverse Reinforcement Learning

- Instead of copying actions directly, the agent infers the underlying reward function that the expert is optimizing, then derives its own policy based on that learned reward.

- Humanoid Example: The humanoid observes humans walking and deduces that the “true” reward function values energy efficiency, maintaining balance, and moving forward steadily. It then uses reinforcement learning to find

Implicit Behaviour Cloning

- A variant of BC that represents the policy implicitly (e.g. as an energy function or using other non-explicit representations) rather than as an explicit mapping from states to actions.

- Humanoid Example: Instead of directly predicting “move knee joint to 45°”, the humanoid learns an energy function where low energy corresponds to poses that look like expert walking. It then moves in ways that minimize this energy.

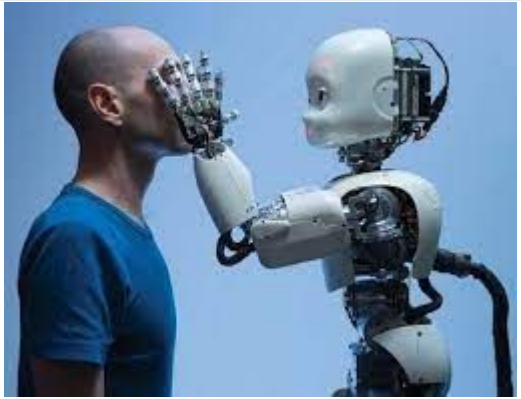
Diffusion Policies

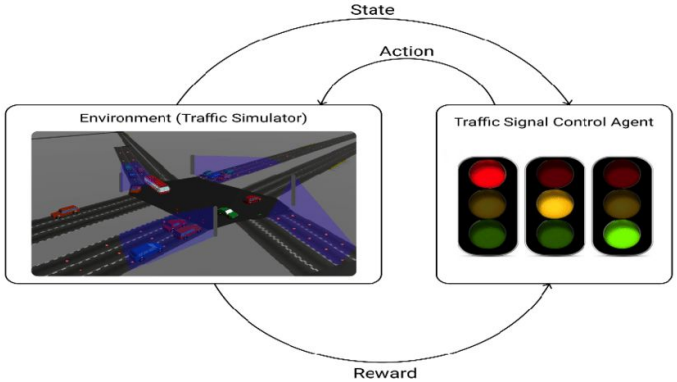
- A recent advanced approach that uses diffusion models (similar to those in AI image generation) to generate complex, multi-modal action sequences.

- Humanoid Example: When the humanoid is in a particular stance, there might be several valid next movements. A diffusion policy can generate multiple possible action sequences that all lead to successful walking, capturing the variability and creativity needed for robust locomotion.

Applications for Reinforcement Learning

1. Gaming industry
2. Robotics
3. Finance
4. Traffic Control
5. Power System
6. Recommendation Systems
7. Healthcare
8. Autonomous Vehicles
9. Supply Chain Management
10. Natural Language Processing
11. … etc





Reference: Google Image, https://emeritus.org/blog/ai-and- ning/

Summary

- In game development, we focus on AI integration in three perspectives:

1. Learn to **play**: Agents can serve as opponents or player allies
2. Learn to **design**: GenAI tools generate 3D models from 2D images or natural language descriptions
3. Learn to **walk**: Agents simulate human-like locomotion as naturally as possible